CalisFacu v1.0

Plan de Aseguramiento de Calidad de Software (SQAP)

Conforme a IEEE Std 730-2002, agrupado según las áreas de IEEE Std 730-2014

Equipo L-001 · COM450 Calidad de Software · Universidad San Francisco Xavier · Gestión II/2026 · Versión del plan 1.2

Tabla 1. Control del documento

Campo	Valor
Producto	CalisFacu v1.0: sistema web de registro de calificaciones para unidades educativas del Subsistema de Educación Regular de Bolivia
Cliente (dueño del producto)	Unidad Educativa Nazareno — directora Fabiana Parrado y profesora Fatima Carreño
Equipo	L-001: Lujan (Líder, Arquitecto, Desarrollador backend) y Galvan (Responsable SQA, Desarrollador frontend / Tester)
Stack	NestJS + TypeScript + TypeORM + PostgreSQL 16 (API) · React + Vite + TypeScript (web) · Jenkins (Multibranch Pipeline) · Render (ADR-0009)
Duración	4 sprints de 1 semana (Semanas 1 a 4)
Versión y fecha	1.2 · 30/10/2026
Estado	Vigente desde la versión 1.0; en ejecución (Sprint 3)
Ubicación	`docs/sqap/SQAP-CalisFacu.docx` y `.pdf` en el repositorio del proyecto

Cómo leer este plan. Cada compromiso de calidad tiene un número, una herramienta que lo hace cumplir y una evidencia que queda archivada. Los números se definen una sola vez, en la Tabla de métricas de la Sección 5 (M01 a M16); las demás secciones se refieren a esas filas. Los datos que el equipo debía completar están en el Anexo F con su estado; desde la versión 1.2 no queda ninguno entre corchetes.

Índice

Agrupación según IEEE Std 730-2014....4
1. Propósito....5
2. Documentos de referencia....6
3. Gestión....7
4. Documentación....8
5. Estándares, prácticas, convenciones y métricas....9
6. Revisiones de software....13
6.1 Inspección formal tipo Fagan (IEEE 1028)....13
6.2 Lista de verificación de la auditoría....14
7. Pruebas....15
7.1 Ambientes....16
7.2 Datos de prueba, regresión y seguridad....16
8. Reporte de problemas y acción correctiva....17
9. Herramientas, técnicas y metodologías....18
10. Control de medios....20
10.1 Repositorio....21
10.2 Estrategia de ramas, pull requests y merge....21
10.3 Mensajes de commit....22
10.4 Versiones y releases....22
10.5 Secretos y control de acceso....23
10.6 Respaldos....23
10.7 Control de cambios de las historias de usuario....23
11. Control de proveedores....24
12. Registros: recolección, mantenimiento y retención....25
13. Capacitación....26
14. Gestión de riesgos....27
15. Glosario....28
16. Procedimiento de cambio e historia del SQAP....30
Anexo A — Requerimientos no funcionales (SRS 1.0)....32
Anexo B — Trazabilidad historia → módulo → pruebas → métricas....32
Anexo C — Configuración que hace cumplir el plan....33
Anexo D — Plantillas....39
Anexo E — Autoevaluación contra la regla del plan verificable....40
Anexo F — Datos y decisiones pendientes....41

Lista de tablas

Tabla 1. Control del documento....1
Tabla 2. Secciones del plan agrupadas por área de IEEE 730-2014....4
Tabla 3. Alcance del plan: historias por sprint....5
Tabla 4. Módulos críticos....5
Tabla 5. Documentos de referencia....6
Tabla 6. Roles y responsabilidades....7
Tabla 7. Matriz RACI de las actividades de calidad....8
Tabla 8. Documentos del proyecto....9
Tabla 9. Métricas de calidad M01 a M16....10
Tabla 10. Dónde se configura cada umbral....11

Tabla 11. Calendario de revisiones....13
Tabla 12. Roles de la inspección Fagan....13
Tabla 13. Etapas de la inspección....14
Tabla 14. Indicadores que verifica cada auditoría....14
Tabla 15. Estrategia de pruebas por nivel....15
Tabla 16. Ambientes de prueba y despliegue....16
Tabla 17. Transiciones permitidas....17
Tabla 18. SLA por severidad....18
Tabla 19. Prioridades....18
Tabla 20. Herramientas del proyecto....19
Tabla 21. Reglas de ramas y pull requests....21
Tabla 22. Política de respaldos....23
Tabla 23. Criterios de selección de dependencias....24
Tabla 24. Dependencias directas principales (las versiones exactas están en package-lock.json)....24
Tabla 25. Proveedores de servicios....25
Tabla 26. Catálogo de evidencias....25
Tabla 27. Competencias y brechas por rol....26
Tabla 28. Plan de capacitación....27
Tabla 29. Matriz de riesgos de calidad....27
Tabla 30. Glosario....29
Tabla 31. Autoría por sección....30
Tabla 32. Historial de versiones del SQAP....30
Tabla 33. Requerimientos no funcionales....32
Tabla 34. Trazabilidad de las historias del alcance....32
Tabla 35. Configuración del job Multibranch Pipeline....33
Tabla 36. Autoevaluación del plan....41
Tabla 37. Pendientes....41

Lista de figuras

Figura 1. Organigrama del proyecto y líneas de independencia....7
Figura 2. Estados del defecto y quién mueve cada transición....17
Figura 3. Etapas del Jenkinsfile y métricas que hace cumplir cada una....20
Figura 4. Recorrido de un cambio: de la tarea a la versión....21

Agrupación según IEEE Std 730-2014

El documento sigue las 16 secciones de IEEE Std 730-2002. Para justificar decisiones se agrupan en las tres áreas de proceso de IEEE Std 730-2014, igual que los entregables de la Parte C de la Práctica 7:

Tabla 2. Secciones del plan agrupadas por área de IEEE 730-2014

Área de IEEE 730-2014	Secciones de este plan	Qué asegura en CalisFacu
Implementación del proceso SQA	1, 2, 3, 15 y 16	Alcance, normas, roles independientes, vocabulario común y control de versiones del propio plan.
Aseguramiento del producto	4, 5 y 7	Que los documentos, el código y el sistema cumplan los RNF con métricas medidas por herramientas.
Aseguramiento del proceso	6, 8, 9, 10, 11, 12, 13 y 14	Que revisiones, defectos, herramientas, repositorio, proveedores, evidencias, capacitación y riesgos funcionen como se planificó.

1. Propósito

Este plan cubre el desarrollo de **CalisFacu v1.0**, sistema web de registro de calificaciones para unidades educativas del Subsistema de Educación Regular de Bolivia. Abarca requerimientos, diseño, construcción, pruebas, despliegue y aceptación de la versión 1.0, durante **4 sprints de 1 semana** de la Gestión II/2026, y lo ejecuta el equipo L-001. El alcance del producto es la pila de producto aprobada por el docente (`backlog_v2_4sprints.xlsx`), más cuatro historias nuevas propuestas en la revisión del backlog.

Tabla 3. Alcance del plan: historias por sprint

Sprint	Historias de usuario incluidas	Resultado verificable
Semana 1	US-17 Autenticación y roles · US-18 Gestión académica base · US-01 Registro central de notas · US-02 Cálculo automático de promedios	Un profesor inicia sesión, registra notas por dimensión y ve la nota trimestral calculada según la RM 001/2026.
Semana 2	US-04 Corrección auditable · US-03 Historial académico · US-19 Registro de asistencia · US-05 Importación desde Excel · US-10 Control de entrega	Toda corrección queda auditada; se importan notas y se registra la asistencia.
Semana 3	US-06 Panel de riesgo · US-07 Boletín en PDF · US-20 Vinculación padre–estudiante · US-11 Consulta de boletín	El profesor genera el boletín con el formato oficial y el padre consulta solo el de sus hijos.
Semana 4	US-09 Centralización de notas · US-08 Alerta de promedio bajo · US-12 Aviso de boletín	La secretaría centraliza el trimestre y los avisos automáticos funcionan.

Roles de usuario: Profesor, Secretaria, Padre de familia y Administrador. **Fuera del alcance:** el módulo de pagos (US-13, US-14, US-15 y US-16), propuesto para una versión posterior. El docente aprobó las historias US-17 a US-20 y dejar los pagos fuera de la v1.0 (Anexo F).

Nivel de integridad: 2 (Moderado) según el esquema de cuatro niveles de IEEE Std 1012. Justificación: una falla (por ejemplo, una nota trimestral mal calculada o un boletín que no se genera) causa una inconveniencia operativa recuperable: la nota se recalcula, la corrección queda auditada y la unidad educativa sigue registrando las notas oficiales en el sistema del Ministerio de Educación. No provoca pérdida de vidas, daño ambiental ni pérdida económica mayor.

Dos condiciones elevarían el nivel a **3 (Mayor)** y obligarían a revisar este plan antes de construir: incluir el módulo de pagos (transacciones económicas) o que CalisFacu reemplace al registro oficial del Ministerio. Como el sistema guarda **datos personales de menores de edad**, se aplican controles de confidencialidad propios de un nivel superior aunque el nivel sea 2: autorización por rol y por vínculo (M09), auditoría de cada cambio (M15) y datos de prueba exclusivamente sintéticos (R07).

Módulos críticos. Son los que modifican o calculan notas oficiales, o deciden quién puede verlas. Los programa solo el Desarrollador backend, se construyen con TDD y exigen cobertura ≥ 90 % (M06):

Tabla 4. Módulos críticos

Módulo crítico	Ruta en el repositorio	Historias	Por qué es crítico
Registro de notas	`api/src/modules/notas`	US-01, US-05	Escribe las notas oficiales, también las importadas.

Módulo crítico	Ruta en el repositorio	Historias	Por qué es crítico
Cálculo de notas	`api/src/modules/calculo`	US-02, US-18 (dimensiones)	Aplica la RM 001/2026; un error cambia quién aprueba.
Corrección auditable	`api/src/modules/auditoria`	US-04	Registro de no repudio de cada cambio de nota.
Autorización	`api/src/modules/auth`	US-17	Decide quién lee o modifica datos de menores.
Vínculo familiar	`api/src/modules/familias`	US-20, US-11 (consulta del padre)	Decide qué padre ve los datos y el boletín de cada menor.

2. Documentos de referencia

Tabla 5. Documentos de referencia

Código	Documento	Versión	Ubicación
BACKLOG	Pila de producto, pila del sprint, criterios de aceptación y reglas del Ministerio (`backlog_v2_4sprints.xlsx`)	2.0	`docs/backlog/`
SRS	Especificación de requerimientos de CalisFacu, derivada del backlog (RNF en el Anexo A)	1.0	`docs/srs/SRS-CalisFacu.pdf`
SAD	Documento de arquitectura: NestJS + TypeORM + PostgreSQL, React; ADR en `docs/adr/`	1.0	`docs/sad/SAD-CalisFacu.pdf`
MATRICES	Libro `matrices-sqap.xlsx`: Métricas, Riesgos, SLA y Componentes de terceros	1.0	`docs/sqap/`
RM 001/2026	Ministerio de Educación: modificación del Reglamento de Evaluación del Desarrollo Curricular (dimensiones Ser 10, Saber 45, Hacer 40, Autoevaluación 5)	05/01/2026	minedu.gob.bo
RM 0190/2024	Ministerio de Educación: Reglamento de Evaluación del Desarrollo Curricular, Educación Regular (evaluación trimestral)	2024	minedu.gob.bo
IEEE 730	IEEE Std 730-2002, Software Quality Assurance Plans; IEEE Std 730-2014, Software Quality Assurance Processes	2002 y 2014	IEEE Xplore (biblioteca USFX)
ISO 25010	ISO/IEC 25010, modelo de calidad del producto (se usa la edición 2011)	2011	Biblioteca USFX
IEEE 1012	IEEE Std 1012, niveles de integridad (Sección 1)	2016	IEEE Xplore (biblioteca USFX)
IEEE 1028	IEEE Std 1028, revisiones, inspecciones y auditorías (Sección 6)	2008	IEEE Xplore (biblioteca USFX)
ISO 29119	ISO/IEC/IEEE 29119, procesos y documentación de pruebas (Sección 7)	2013–2022	Biblioteca USFX
ISO 29110	ISO/IEC 29110, perfil para entidades muy pequeñas (equipo de dos personas)	2011	Biblioteca USFX
OWASP	OWASP Top 10 y OWASP ASVS, ediciones vigentes (Secciones 5, 7 y 11)	Vigente	owasp.org

Código	Documento	Versión	Ubicación
CC	Conventional Commits (mensajes de commit, Sección 10)	1.0.0	conventionalcommits.org
SemVer	Semantic Versioning (versiones y etiquetas, Sección 10)	2.0.0	semver.org
GUÍAS	Guías de las Prácticas 1 a 8 de COM450 (pruebas unitarias, TDD, SonarQube, Cypress, JMeter, Jenkins, SQAP, CI/CD con GitHub Actions)	II/2026	Classroom de COM450

3. Gestión

El equipo L-001 tiene **dos integrantes**, conformación aceptada para la práctica aunque la guía sugiere 3 o 4; por eso cada uno asume más de un rol y la independencia se compensa como se explica más abajo. **Lujan** es Líder de proyecto, Arquitecto y Desarrollador backend; **Galvan** es Responsable SQA y Desarrollador frontend / Tester. El cliente es la Unidad Educativa Nazareno, representada por la directora Fabiana Parrado y la profesora Fatima Carreño, que actúan como dueños del producto.

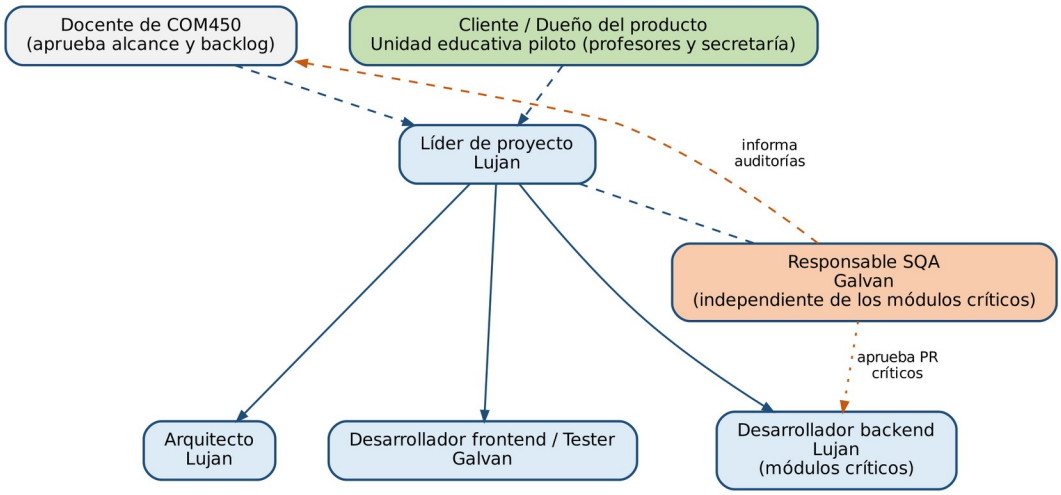


Figura 1. Organigrama del proyecto y líneas de independencia

Regla de independencia: quien produce un artefacto no lo revisa ni lo aprueba. Galvan **no programa** los módulos críticos de la Tabla 4; los escribe Lujan y Galvan revisa sus commits (ADR-0008). El código de Galvan (interfaz, API no crítica, `Jenkinsfile` y scripts de Cypress y JMeter) lo revisa Lujan. El Responsable SQA tiene autoridad para rechazar cualquier entregable que no cumpla los objetivos M01 a M16 y para detener un release.

Tabla 6. Roles y responsabilidades

Rol	Responsabilidad principal	Integrante
Líder de proyecto	Coordina el equipo y el cronograma, comunica con el docente y el cliente, fija la prioridad de los defectos, aprueba cambios de historias y rinde cuentas por el proyecto.	Lujan
Arquitecto	Define la arquitectura NestJS y el modelo de datos PostgreSQL, registra las decisiones como ADR y mantiene el manual técnico.	Lujan

Rol	Responsabilidad principal	Integrante
Responsable SQA	Mantiene el SQAP y `matrices-sqap.xlsx`, aprueba los PR de los módulos críticos, modera las inspecciones y audita el cumplimiento del plan. No programa módulos críticos.	Galvan
Desarrollador frontend / Tester	Construye la interfaz React y la API no crítica, mantiene el pipeline y ejecuta las pruebas E2E (Cypress), de carga (JMeter) y de usabilidad.	Galvan
Desarrollador backend	Desarrolla los módulos críticos con TDD, las entidades de TypeORM, las migraciones y las pruebas Jest de la API.	Lujan

Independencia parcial y cómo se compensa. Con dos personas la independencia es parcial en tres puntos, y cada uno tiene una compensación:

- Galvan es a la vez Responsable SQA y Tester: las evidencias de prueba que produce (informes de Cypress, JMeter y usabilidad) las audita **Lujan** en la auditoría de cierre.
- Galvan mantiene el pipeline que mide su propio código: todo cambio en el `Jenkinsfile` y en `jenkins/` lo aprueba Lujan, y las ramas `develop` y `main` exigen el check de Jenkins aunque el autor sea administrador.
- Las inspecciones Fagan del SRS y del SAD incluyen como inspector a un integrante de otro equipo del curso (Sección 6).

Matriz RACI (R ejecuta, A rinde cuentas, C es consultado, I es informado; exactamente una A por fila). Lujan cubre las columnas Líder, Arquitecto y Dev; Galvan cubre SQA y Tester.

Tabla 7. Matriz RACI de las actividades de calidad

Actividad	Líder	Arquitecto	SQA	Dev	Tester
Redactar y mantener el SQAP	A	C	R	I	C
Inspeccionar el SRS (autor: Líder)	C	C	A	I	R
Inspeccionar el SAD (autor: Arquitecto)	I	C	A/R	I	C
Revisar código de módulos críticos (autor: Dev)	I	C	A/R	C	I
Revisar código de interfaz, CI y scripts de prueba (autor: Tester)	I	A	I	R	C
Ejecutar pruebas de carga	I	A	C	I	R
Aprobar un cambio de historia de usuario	A/R	C	C	I	I
Aprobar un release (revisor obligatorio de producción)	A	C	R	I	C
Probar la restauración de respaldos	I	C	A	R	I
Auditar el cumplimiento del plan	R	I	A/R	I	C

Comprobación de independencia: en cada fila donde un integrante es autor, la A o la R de la revisión recae en el otro. En la auditoría del plan, Lujan (R) audita las evidencias que produjo Galvan y Galvan (A/R) audita las de desarrollo.

4. Documentación

Documentos que el proyecto produce. Todos se versionan en `docs/` del repositorio y se modifican por pull request.

Tabla 8. Documentos del proyecto

Documento	Responsable	Fecha prevista	Criterio de adecuación	Tipo de revisión
SRS	Líder (Lujan)	Semana 1, día 1	Cada historia del alcance tiene criterios Dado/Cuando/Entonces; cada RNF del Anexo A tiene métrica y valor verificable; las reglas de la RM 001/2026 tienen la tabla de casos firmada por el cliente; 0 preguntas abiertas (❓).	Inspección Fagan con el cliente y un inspector externo (Sección 6).
SAD y ADR	Arquitecto (Lujan)	Semana 1, día 2	Modelo de datos (entidades TypeORM) normalizado; módulos NestJS con sus dependencias; decisiones que cubren RNF-03 (cola de PDF), RNF-04 (guards y auditoría) y RNF-07 (respaldos); un ADR por decisión.	Inspección Fagan (Sección 6).
Pila de producto y del sprint	Líder (Lujan)	Semana 1, día 1; se actualiza cada día 1	Cumple la Definition of Ready; cada tarea tiene responsable, revisor y horas.	Revisión del backlog con el cliente cada día 5.
Contrato de la API (OpenAPI)	Dev (Lujan)	Se genera en cada build	Todo endpoint aparece en `/api/docs` con DTO de entrada y salida y respuestas 401/403/422.	Revisión del PR.
Manual de usuario	Tester (Galvan)	Semana 4, día 3	Cubre las tareas de los cuatro roles con capturas; un profesor piloto registra las notas de una actividad usando solo el manual.	Walkthrough con el profesor piloto; revisa Lujan.
Manual técnico	Arquitecto (Lujan)	Semana 4, día 3	Galvan levanta el sistema en una máquina limpia siguiendo solo el manual, en 60 minutos o menos.	Walkthrough; revisa Galvan.
Informe de pruebas	Tester (Galvan)	Cada día 5 (semanas 1 a 4)	Resultados de Jest, Cypress, JMeter y usabilidad contra M01 a M16, con enlaces a los runs.	Auditoría de cierre; lo audita Lujan porque el autor es Galvan.
Actas	Responsable SQA (Galvan)	En cada revisión, inspección, auditoría y aceptación	Fecha, participantes, artefacto y versión, hallazgos con severidad, decisión y firmas.	Firma de los participantes.
CHANGELOG	Líder (Lujan)	En cada release	Una entrada por versión, agrupada por tipo de commit.	Revisión del PR de release.

5. Estándares, prácticas, convenciones y métricas

Estándar de codificación: TypeScript en modo `strict` con **typescript-eslint** (configuración `recommended-type-checked`) y **Prettier** para el formato, en la API y en la web. Las reglas de estilo que la herramienta no fija se toman de la *Airbnb JavaScript Style Guide*. **Justificación:** un solo lenguaje y un solo conjunto de reglas para todo el repositorio; las dos herramientas se ejecutan en el pre-commit y en el CI, de modo que el estándar se hace cumplir automáticamente (M16) y no depende de la revisión manual.

Convenciones de nombres (los términos del dominio, en español, según el glosario de la Sección 15):

- **Clases, DTO y módulos:** PascalCase (ejemplo: `CalculoService`, `RegistrarNotaDto`).
- **Variables y métodos:** camelCase (ejemplo: `calcularNotaTrimestral()`).
- **Archivos:** kebab-case con sufijo de NestJS (ejemplo: `calculo.service.ts`, `notas.controller.ts`).
- **Tablas y columnas de PostgreSQL:** snake_case mediante la estrategia de nombres `SnakeNamingStrategy` de TypeORM; tablas en plural en `@Entity()` (ejemplo: `notas`, `registros_auditoria`).
- **Endpoints:** kebab-case, sustantivos en plural, prefijo `/api` (ejemplo: `/api/notas`, `/api/boletines/:id/pdf`).
- **Pruebas:** `*.spec.ts` junto al código (unitarias), `*.e2e-spec.ts` en `api/test` (integración HTTP), `*.cy.ts` en `e2e/` (Cypress); cada caso empieza con "debe..." (ejemplo: `it('debe redondear 50,5 a 51 y aprobar')`).
- **Ramas y commits:** Conventional Commits con referencia a la historia (Sección 10).

Métricas (detalle en la hoja Métricas de `matrices-sqap.xlsx`). Es la única tabla donde se definen los umbrales del plan:

Tabla 9. Métricas de calidad M01 a M16

Característica 25010	Métrica y fórmula	Instrumento, condiciones y momento	Objetivo	RNF / riesgo	Responsable
Adecuación funcional (corrección)	M01 · % de casos de la tabla de cálculo firmada por el cliente que pasan = casos en verde / casos de la tabla	Jest (P1, P2), suite `calculo.casos.spec.ts`; en cada build de Jenkins (PR y main)	100 %	RNF-01, R02	Dev (Lujan)
Adecuación funcional (completitud)	M02 · % de escenarios E2E críticos que pasan = escenarios en verde / escenarios	Cypress (P4) contra staging con datos sintéticos; en cada build de develop	100 %	RNF-02	Tester (Galvan)
Eficiencia de desempeño (comportamiento temporal)	M03 · p95 del tiempo de respuesta de "generar boletín de un estudiante" = percentil 95	JMeter (P5): 100 usuarios concurrentes, 10 min, ambiente de rendimiento (agente de Jenkins); en cada RC	≤ 2000 ms	RNF-03	Tester (Galvan)
Eficiencia de desempeño (capacidad)	M04 · Tasa de error bajo carga de "registrar planilla" y "generar boletín" = peticiones fallidas / peticiones	JMeter (P5), mismas condiciones que M03; en cada RC	< 1 % en cada operación	RNF-03, R05	Tester (Galvan)
Mantenibilidad (capacidad de prueba)	M05 · Cobertura de líneas del código nuevo = líneas nuevas cubiertas / líneas nuevas	Jest (lcov) + SonarQube Cloud (P3); en cada build de develop	≥ 80 %	RNF-05, R03	SQA (Galvan)

Característica 25010	Métrica y fórmula	Instrumento, condiciones y momento	Objetivo	RNF / riesgo	Responsable
Mantenibilidad (capacidad de prueba)	M06 · Cobertura de líneas de cada módulo crítico = líneas cubiertas / líneas del módulo	Jest `coverageThreshold` por ruta (P1, P2); en cada PR	≥ 90 % por módulo	RNF-05, RNF-01	SQA (Galvan)
Mantenibilidad (modificabilidad)	M07 · Líneas duplicadas en código nuevo = duplicadas / nuevas	SonarQube Cloud (P3); en cada build de develop	≤ 3 %	RNF-05	SQA (Galvan)
Mantenibilidad (analizabilidad)	M16 · Errores y advertencias de lint y formato = conteo	ESLint `--max-warnings=0` y `prettier --check`; pre-commit y cada PR	0	RNF-05	SQA (Galvan)
Seguridad (confidencialidad)	M08 · Vulnerabilidades altas o críticas = conteo en código nuevo y en dependencias	SonarQube Cloud (Security Rating) + `npm audit --audit-level=high` + Dependabot; SonarQube en cada build de develop, npm audit en cada build	0	RNF-04, R06	SQA (Galvan)
Seguridad (integridad)	M09 · % de rutas protegidas que rechazan con HTTP 403 a un usuario sin permiso = rutas que rechazan / rutas protegidas	Jest + Supertest contra PostgreSQL, suite `autorizacion.e2e-spec.ts` (rol, asignación profesor–materia y vínculo padre–estudiante); en cada PR	100 %	RNF-04	Dev (Lujan)
Seguridad (no repudio)	M15 · % de cambios de nota con registro de auditoría completo (quién, cuándo, valor anterior, valor nuevo, motivo, origen)	Jest + Supertest, suite `auditoria.e2e-spec.ts`; en cada PR	100 %	RNF-04	Dev (Lujan)
Usabilidad (aprendizaje)	M11 · Tasa de éxito de la tarea "registrar las notas de una actividad" sin ayuda = profesores que la completan / profesores	Prueba con 5 profesores que nunca usaron el sistema; Semana 3, día 4	≥ 80 % (4 de 5)	RNF-06	Tester (Galvan)
Usabilidad (operabilidad)	M12 · Puntaje SUS promedio de los mismos 5 profesores	Cuestionario SUS de 10 ítems al terminar M11	≥ 68	RNF-06	Tester (Galvan)
Fiabilidad (disponibilidad)	M13 · Disponibilidad mensual de producción = (tiempo total − tiempo caído) / tiempo total	UptimeRobot cada 5 min sobre /api/salud; desde el release v1.0	≥ 99,0 %	RNF-07	SQA (Galvan)
Fiabilidad (recuperabilidad)	M14 · RPO y RTO de la base de datos	Respaldo diario cifrado (`backup.yml` en GitHub Actions); prueba de restauración cada día 5	RPO ≤ 24 h; RTO ≤ 2 h	RNF-07, R10	Dev (Lujan)

Seis características de ISO/IEC 25010 con al menos dos métricas cada una. M01 a M09 conservan el número que tenían en la versión 0.2 del plan y las métricas nuevas van de M11 a M16 (M10 no se asigna); así un identificador nunca cambia de significado entre versiones.

El mismo número en dos lugares. Cada umbral de la Tabla 9 está configurado en la herramienta que lo hace cumplir. Si un número cambia, se cambia aquí, en `matrices-sqap.xlsx` y en la configuración, en el mismo pull request (Sección 16).

Tabla 10. Dónde se configura cada umbral

Métrica	Dónde se hace cumplir	Configuración
M01 · 100 %	`Jenkinsfile`, etapa Pruebas › API	`npm run -w api test:cov` + paso `junit`: el build falla si un caso falla
M02 · 100 %	`Jenkinsfile`, etapa E2E (rama develop)	`npx cypress run`: código de salida distinto de 0 si falla un escenario
M03 · ≤ 2000 ms	`Jenkinsfile`, etapa Rendimiento (etiquetas `v*-rc.*`)	`jq -e ".Total.pct2ResTime <= 2000" perf/boletin/statistics.json`
M04 · < 1 %	`Jenkinsfile`, etapa Rendimiento	`jq -e ".Total.errorPct < 1"` sobre `perf/boletin/` y `perf/planilla/`
M05 · ≥ 80 %	Quality Gate "Sonar way" de SonarQube Cloud	Coverage on New Code is less than 80.0
M06 · ≥ 90 %	`api/jest.config.ts`	`coverageThreshold` con `lines: 90` para cada ruta de la Tabla 4
M07 · ≤ 3 %	Quality Gate "Sonar way"	Duplicated Lines (%) on New Code is greater than 3.0
M08 · 0	Quality Gate + etapa Dependencias	Security Rating on New Code is worse than A · `npm audit --audit-level=high`
M09 · 100 %	`Jenkinsfile`, etapa Pruebas › API	Suite `autorizacion.e2e-spec.ts` dentro de `npm run -w api test:cov`
M11, M12	Acta de la prueba de usabilidad	Planilla de resultados en `docs/evidencias/usabilidad/`
M13 · ≥ 99,0 %	UptimeRobot	Intervalo de 5 minutos sobre `/api/salud`; alerta por correo a ambos integrantes
M14 · 24 h / 2 h	`backup.yml` (GitHub Actions)	cron `0 7 * * *` (03:00 en Bolivia) y job `restaurar` cada día 5
M15 · 100 %	`Jenkinsfile`, etapa Pruebas › API	Suite `auditoria.e2e-spec.ts` dentro de `npm run -w api test:cov`
M16 · 0	`Jenkinsfile`, etapa Lint + pre-commit	`eslint . --max-warnings=0` y `prettier --check .`

Fragmentos de configuración (archivos completos en el Anexo C):

```
// api/jest.config.ts (fragmento): M06, cobertura de los módulos críticos
coverageThreshold: {
  './src/modules/notas/':     { lines: 90 },
  './src/modules/calculo/':   { lines: 90 },
  './src/modules/auditoria/': { lines: 90 },
  './src/modules/auth/':      { lines: 90 },
  './src/modules/familias/':  { lines: 90 },
},

// Jenkinsfile, etapa Rendimiento (fragmento): M03 y M04
jq -e ".Total.pct2ResTime <= 2000" perf/boletin/statistics.json
jq -e ".Total.errorPct < 1"         perf/boletin/statistics.json
jq -e ".Total.errorPct < 1"         perf/planilla/statistics.json

Quality Gate "Sonar way" (SonarQube Cloud):
```

```
  Coverage on New Code is less than 80.0          -> falla (M05)
  Duplicated Lines (%) on New Code is greater than 3.0 -> falla (M07)
  Security Rating on New Code is worse than A     -> falla (M08)
```

El percentil 95 corresponde a `pct2ResTime` porque el dashboard de JMeter calcula por defecto los percentiles 90, 95 y 99; `jq -e` termina con error, y la etapa falla, si la condición es falsa. Los planes `.jmx` leen la carga con `${__P(usuarios)}` y `${__P(duracion)}`. A diferencia de la versión 0.2, la tasa de error se exige también en el boletín: una respuesta 500 rápida no debe contar como una respuesta rápida.

6. Revisiones de software

Calendario de revisiones en el ciclo de vida. "Día" es el día hábil dentro de cada sprint de una semana (día 1 = lunes).

Tabla 11. Calendario de revisiones

Revisión	Artefacto	Cuándo	Quién revisa	Lista de verificación	Criterio de aceptación
Revisión de requerimientos e inspección Fagan del SRS	SRS, backlog y tabla de casos de cálculo	Semana 1, días 1 y 2	Moderador y escriba: Galvan. Inspectores: profesor del cliente y un integrante de otro equipo. Autor: Lujan (responde, no aprueba).	Cada historia con criterios Dado/Cuando/Entonces; cada RNF con métrica y valor; reglas de la RM 001/2026 con casos de ejemplo; términos del glosario; 0 ❓ abiertos.	Acta firmada por el cliente; 0 defectos mayores abiertos; tabla de casos firmada.
Inspección Fagan del SAD	SAD y ADR	Semana 1, día 2	Moderador y escriba: Galvan. Lector e inspector: integrante de otro equipo. Autor: Lujan.	Modelo de datos normalizado con índices para notas y boletines; módulos críticos aislados; cola para PDF; guards de autorización; respaldos; secretos fuera del código.	0 defectos mayores abiertos; acta en `docs/actas/`.
Revisión de diseño detallado	Diseño de US-04 y US-05 (semana 2); de la cola de boletines de US-07 (semana 3)	Día 1 del sprint	Galvan (autor: Lujan)	Transacciones y auditoría de solo inserción; validación por fila de la importación; reintentos y aviso de la cola; pruebas previstas por criterio.	Acta con decisiones; ningún hallazgo mayor abierto al iniciar la construcción.
Revisión de código	Todo commit en develop; el PR de release develop → main	1 día hábil después del push; en cada PR de release	Módulos críticos y API: Galvan. Interfaz, CI y scripts de prueba: Lujan. Nadie aprueba su propio código.	Build de Jenkins en verde; DoD; pruebas nuevas para cada criterio tocado; sin secretos; nombres según la Sección 5.	Comentario de revisión en el commit; PR de release aprobado (ADR-0008).
Revisión del sprint (demo)	Incremento en staging	Día 5 de cada sprint	Cliente (directora y profesora) con el equipo	Cada historia del sprint contra sus criterios de aceptación.	Acta con las historias aceptadas y los cambios solicitados.
Auditoría de cierre de sprint	Informe de pruebas, evidencias y SQAP	Día 5 de cada sprint	Galvan audita los artefactos de desarrollo; Lujan audita las evidencias de prueba que produjo Galvan.	Siete indicadores del plan verificable y doce del build sano (Tabla 14); M01 a M16 con evidencia.	Acta firmada por ambos; hallazgos como issues con responsable.

Revisión	Artefacto	Cuándo	Quién revisa	Lista de verificación	Criterio de aceptación
Auditoría final	SQAP, evidencias y release candidate RC2	Semana 4, día 5	Igual que la auditoría de cierre, más el docente como observador	Igual que la auditoría de cierre, más la autoevaluación del Anexo E.	Release v1.0 aprobado según la RACI.

6.1 Inspección formal tipo Fagan (IEEE 1028)

Tabla 12. Roles de la inspección Fagan

Rol	Quién	Responsabilidad
Moderador	Galvan (SQA)	Planifica, verifica criterios de entrada, conduce la reunión y decide el seguimiento.
Autor	Lujan	Entrega el artefacto, aclara dudas en la reunión y hace el retrabajo. No inspecciona su propio artefacto.
Lector	Integrante de otro equipo	Parafrasea el artefacto sección por sección durante la reunión.
Inspectores	Integrante de otro equipo; profesor del cliente (solo en el SRS); Galvan	Preparan individualmente y registran defectos en la planilla de inspección.
Escriba	Galvan	Registra cada defecto con ubicación, severidad (mayor o menor) y decisión.

Tabla 13. Etapas de la inspección

Etapa	Qué se hace	Duración máxima
Planificación	El moderador verifica que el artefacto esté completo, arma la planilla y convoca.	30 min
Visión general	El autor explica el contexto del artefacto.	20 min
Preparación individual	Cada inspector revisa con la lista de verificación de la Tabla 11 a un ritmo de 10 páginas por hora o menos.	1 h por inspector
Reunión de inspección	El lector parafrasea; se registran defectos; no se discuten soluciones.	2 h
Retrabajo	El autor corrige cada defecto registrado.	1 día hábil
Seguimiento	El moderador verifica cada corrección y cierra el acta.	30 min

Métricas de la inspección (se registran en el acta): tiempo de preparación por inspector; ritmo de preparación (páginas / hora, objetivo ≤ 10); densidad de defectos (defectos / página); defectos mayores y menores; eficacia de la inspección = defectos encontrados en la inspección / (defectos encontrados en la inspección + defectos del mismo artefacto encontrados después). Criterio de salida: 0 defectos mayores abiertos.

6.2 Lista de verificación de la auditoría

Tabla 14. Indicadores que verifica cada auditoría

Regla	Indicador	Se cumple si…
Plan verificable (P7)	Completo	Están las 16 secciones; las que no aplican lo justifican.
	Medible	M01 a M16 tienen fórmula y valor numérico.
	Instrumentado	Cada métrica nombra herramienta, condiciones y momento.
	Asignado e independiente	Ningún autor aprobó su propio artefacto (historial de PR y actas).
	Consistente	Cada umbral de la Tabla 9 aparece igual en la Tabla 10, en `matrices-sqap.xlsx` y en la configuración.
	Trazable	Cada métrica cita un RNF o riesgo; cada riesgo alto tiene mitigación con responsable.
	Evidenciable	Cada evidencia de la Sección 12 existe en su ubicación.
Build sano (P6) y pipeline sano (P8)	Resultado	SUCCESS en el último build de main, en la Stage View.
	Pruebas ejecutadas	Más de cero y las mismas que corren en local (Test Result del build).
	Pruebas fallidas	0 fallos y 0 errores; el informe JUnit está publicado (Test Result Trend).
	Artefacto	El paquete del build está archivado y asociado al build (Build Artifacts).
	Reproducibilidad	Todo el pipeline está en el `Jenkinsfile`; el job solo tiene la URL del repositorio.
	Cobertura	Exigida por la herramienta (M05, M06), no solo medida.
	E2E	Cypress ejecutó todos sus escenarios y pasaron (M02).
	Orden	Las etapas de despliegue están después de las de prueba y solo corren si estas pasan.
	Disparo automático	Un push o un PR inicia el build sin clic en 3 minutos o menos (escaneo del Multibranch).
	Verificación posterior	El smoke test pasa y el commit desplegado es el del build.
	Secretos	Ninguno en el repositorio ni impreso en la consola de Jenkins.
	Hooks	Husky activo: un mensaje fuera de Conventional Commits se rechaza en local.

7. Pruebas

Estrategia por nivel. Cada criterio de aceptación de la hoja Criterios de aceptación indica la prueba que lo verifica; la trazabilidad completa está en el Anexo B.

Tabla 15. Estrategia de pruebas por nivel

Nivel	Técnica y herramienta	Responsable	Criterio de entrada	Criterio de salida
Unitario	TDD obligatorio en los cinco módulos críticos; Jest. Aritmética decimal (decimal.js) en el cálculo.	Dev (Lujan); API no crítica: Galvan	Historia que cumple la Definition of Ready	100 % de casos en verde (M01); cobertura ≥ 90 % por módulo crítico (M06) y ≥ 80 % en código nuevo (M05)
Integración	Jest + Supertest contra PostgreSQL 16 real (service container en CI, Docker en local), con las migraciones de TypeORM aplicadas desde cero en cada ejecución	Dev (Lujan)	Unitarias en verde	100 % de suites en verde; 100 % de rutas protegidas rechazan con 403 (M09); 100 % de cambios auditados (M15)
Sistema	E2E con Cypress contra staging (M02); carga con JMeter en el ambiente de rendimiento (M03, M04); análisis de seguridad (M08)	Tester (Galvan)	Smoke test en verde con el commit correcto	100 % de escenarios E2E; p95 del boletín ≤ 2000 ms; tasa de error < 1 %; 0 vulnerabilidades altas o críticas
Aceptación	Escenarios del cliente sobre las historias del sprint; prueba de usabilidad con 5 profesores nuevos y SUS (M11, M12)	Líder (Lujan) con el cliente; usabilidad: Tester	Pruebas de sistema aprobadas	0 defectos S1 o S2 abiertos; ≥ 4 de 5 profesores completan la tarea; SUS ≥ 68; acta firmada

Cobertura mínima global y por módulo crítico: código nuevo de todo el repositorio ≥ 80 % de líneas, medido por SonarQube Cloud (M05); cada módulo crítico de la Tabla 4 ≥ 90 % de líneas, exigido por Jest (M06).

La integración usa PostgreSQL real y no una base en memoria ni SQLite, porque las diferencias de tipos (`numeric`, `timestamptz`), collation y restricciones dejarían pasar defectos que solo aparecen en producción.

7.1 Ambientes

Tabla 16. Ambientes de prueba y despliegue

Ambiente	Para qué	Dónde	Datos
Local	Desarrollo y pruebas rápidas	Docker Compose: PostgreSQL 16 y Redis	Seed sintético (`npm run -w api seed`)
CI	Pruebas de cada PR y de main	Agente de Jenkins con PostgreSQL 16 y Redis del `jenkins/docker-compose.yml`; una base por build	Base vacía + migraciones + fixtures; se borra al terminar
Staging	E2E, smoke test y demo del sprint	API y web en Render (render.yaml), PostgreSQL de Render	Seed sintético; se recrea en cada sprint
Rendimiento	JMeter (M03, M04)	API compilada del commit etiquetado, ejecutada en el agente de Jenkins con su PostgreSQL y Redis	Seed sintético de 40 cursos y 60 estudiantes por curso
Producción	Uso del cliente	Igual que staging, con su propia base y secretos	Datos reales; nunca se copian a otro ambiente

El servicio gratuito de Render se suspende tras 15 minutos sin tráfico y tarda cerca de un minuto en despertar. Por eso la carga se mide en el ambiente de rendimiento (agente de Jenkins) y no en staging. Cada plan de JMeter inicia sesión en un grupo de preparación que no se mide y sube a los 100 usuarios en 60 segundos.

7.2 Datos de prueba, regresión y seguridad

- **Datos:** solo sintéticos, generados con `@faker-js/faker` (locale `es`) en seeds y factories versionados. Está prohibido copiar datos reales a cualquier ambiente que no sea producción (R07).
- **Regresión:** la suite completa (unitaria, integración y E2E) corre en cada merge a develop; cada defecto corregido agrega una prueba que lo reproduce (Sección 8).
- **Seguridad:** autorización (M09) y auditoría (M15) en cada PR; análisis estático y de dependencias (M08); revisión manual contra el OWASP Top 10 en la auditoría de la semana 3.
- **Usabilidad:** cinco profesores que nunca usaron el sistema, sin ayuda, con un observador que solo registra; al terminar, el cuestionario SUS. Lo conduce Galvan y lo audita Lujan.

8. Reporte de problemas y acción correctiva

Estados del defecto y transiciones: todo defecto sigue el ciclo **Nuevo** → **Asignado** → **En progreso** → **Resuelto** → **Verificado** → **Cerrado**. Si la prueba de confirmación falla, pasa de Resuelto a **Reabierto**; un defecto Cerrado que reaparece también pasa a Reabierto. Desde Reabierto, el Líder lo vuelve a Asignado.

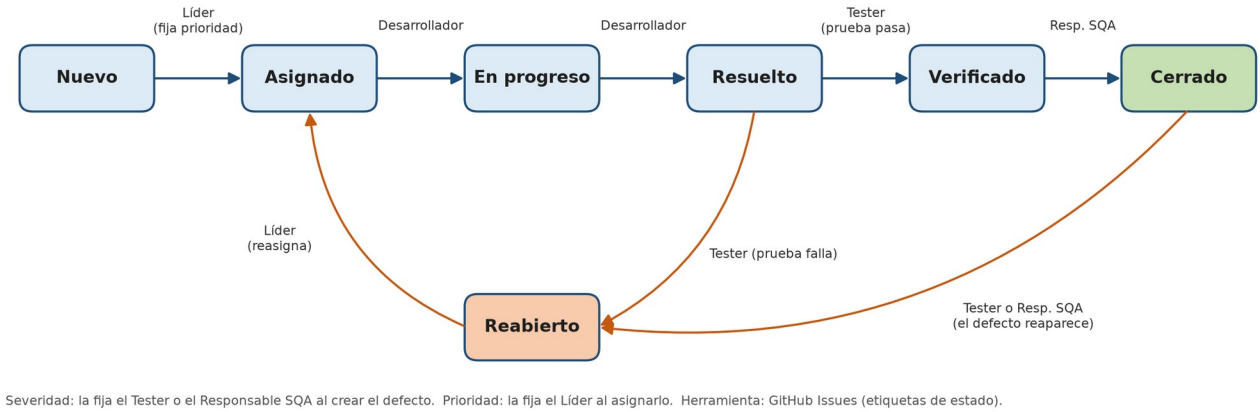


Figura 2. Estados del defecto y quién mueve cada transición

Tabla 17. Transiciones permitidas

Transición	Quién la mueve	Condición
→ Nuevo	Tester, cualquier integrante o el cliente	Issue creado con la plantilla de defecto y severidad propuesta.
Nuevo → Asignado	Líder	Fija la prioridad y el responsable.
Asignado → En progreso	Desarrollador asignado	Trabaja en develop; sus commits citan Refs: CAL-NNN.
En progreso → Resuelto	Desarrollador asignado	PR fusionado con la prueba que reproduce el defecto.
Resuelto → Verificado	Tester (Galvan). **Si el defecto lo resolvió Galvan, verifica Lujan.**	La prueba de confirmación pasa en staging.
Verificado → Cerrado	Responsable SQA (Galvan). Si Galvan resolvió el defecto, cierra Lujan.	Evidencia enlazada en el issue.

Transición	Quién la mueve	Condición
Resuelto o Cerrado → Reabierto	Tester o Responsable SQA	La prueba de confirmación falla o el defecto reaparece.

Diferencia entre severidad y prioridad:

- **Severidad:** cuánto daño causa el defecto en el sistema (S1 a S4). La fija el Tester o el Responsable SQA al registrarlo.
- **Prioridad:** con qué urgencia se corrige frente a otras tareas (P1 a P3). La fija el Líder según el calendario del cliente; por ejemplo, el cierre de un trimestre.

SLA de resolución de defectos. Los tiempos se cuentan en horario hábil (lunes a viernes, 8:00 a 18:00). El equipo no tiene guardia de fin de semana, salvo en la semana de cierre de trimestre del cliente, cuando Lujan queda de guardia para S1.

Tabla 18. SLA por severidad

Severidad	Definición	Respuesta	Resolución
Crítica (S1)	El sistema es inusable, hay pérdida o corrupción de notas, o un usuario accede a datos que no le corresponden (ej. una nota trimestral se guarda con un valor incorrecto; un padre ve el boletín de otro estudiante).	4 horas hábiles	1 día hábil
Alta (S2)	Funcionalidad principal afectada sin alternativa y sin pérdida de datos (ej. el boletín PDF falla en cursos de más de 50 estudiantes, CAL-001).	1 día hábil	2 días hábiles
Media (S3)	Funcionalidad secundaria afectada o existe alternativa (ej. el filtro por trimestre del historial falla, pero el estudiante se encuentra por nombre).	1 día hábil	1 semana
Baja (S4)	Defecto cosmético o mejora menor (ej. texto desalineado en el pie del boletín).	3 días hábiles	Siguiente sprint

Tabla 19. Prioridades

Prioridad	Significado	Cuándo se usa
P1	Se corrige antes que cualquier tarea del sprint.	Todo S1; un S2 que bloquea el cierre de trimestre o la demo.
P2	Entra en el sprint en curso.	S2 restantes; S3 que afectan a una historia del sprint.
P3	Se planifica en el siguiente sprint o en la Release 2.	S3 restantes y S4.

Herramienta de gestión: GitHub Issues con el formulario `.github/ISSUE_TEMPLATE/defecto.yml` (Anexo D), etiquetas `estado:*`, `sev:S1` a `sev:S4` y `prio:P1` a `prio:P3`, y el tablero de GitHub Projects del equipo. Los defectos se numeran CAL-001, CAL-002…, número que nunca se reutiliza.

Acción correctiva: todo defecto S1 o S2 registra su causa raíz (técnica de los cinco porqués) en el issue antes de cerrarse; todo defecto corregido agrega una prueba automatizada que lo reproduce; todo S1 produce además

un postmortem sin culpables en `docs/actas/postmortem-CAL-NNN.md`, con acciones, responsable y fecha. Si dos defectos de la misma causa aparecen en un sprint, la causa se registra como riesgo en la Sección 14.

Ejemplo (simulado): CAL-001, "El boletín en PDF responde HTTP 500 en cursos de más de 50 estudiantes". Severidad S2 (función principal sin alternativa, sin pérdida de datos); prioridad P1 porque bloquea el cierre de trimestre; relacionado con RNF-03, M03, M04 y el riesgo R05. Detectado por la etapa Rendimiento del build de la etiqueta RC1; se corrige con la cola de la tarea T-046 de la pila del sprint.

9. Herramientas, técnicas y metodologías

Tabla 20. Herramientas del proyecto

Propósito	Herramienta	Versión	Práctica del curso
Requerimientos y tablero	GitHub Projects + Issues; `backlog_v2_4sprints.xlsx` como entregable	—	—
Control de versiones	Git + GitHub (Git Flow: develop y main protegidas)	Git ≥ 2.40	6, 8
Hooks locales	Husky + lint-staged + commitlint	Fijada en `package-lock.json`	8
Lenguaje y runtime	TypeScript (`strict`) sobre Node.js	Node 22 LTS	8
API	NestJS, TypeORM, PostgreSQL	PostgreSQL 16; resto fijado en `package-lock.json`	—
Web	React + Vite	Fijada en `package-lock.json`	8
Integración y despliegue continuos	Jenkins (Multibranch Pipeline, `Jenkinsfile` en la raíz), en Docker con la imagen de `jenkins/Dockerfile`	LTS, JDK 17	6, 8
Respaldos programados	GitHub Actions, solo `backup.yml`	—	8
Análisis estático	SonarQube Cloud, Quality Gate "Sonar way"; ESLint + Prettier	—	3
Pruebas unitarias y de integración	Jest + Supertest (API); Vitest (web)	Fijada en `package-lock.json`	1, 2
Pruebas E2E	Cypress	Fijada en `package-lock.json`	4
Pruebas de carga	Apache JMeter	5.6.3	5
Dependencias y licencias	`npm audit`, Dependabot, `license-checker`	—	3
Hosting	Render (API, web estática, PostgreSQL, Redis), definido en render.yaml	Planes gratuitos	8
Monitoreo	UptimeRobot (plan gratuito): monitor HTTP de /api/salud de staging y de producción	—	—

Propósito	Herramienta	Versión	Práctica del curso
Gestión de defectos y comunicación	GitHub Issues; grupo de WhatsApp (avisos urgentes, S1) y servidor de Discord (coordinación diaria)	—	—

Por qué Jenkins. Es la herramienta de integración continua de la Práctica 6 y el equipo ya la domina; el `Jenkinsfile` aplica los mismos conceptos de la Práctica 8 (etapas en orden, despliegue solo después de las pruebas, smoke test, secretos fuera del código). Se configura como **Multibranch Pipeline** y no como un job Pipeline simple, porque solo así existen `BRANCH_NAME`, los builds de pull requests y los de etiquetas: con un job simple, `when { branch 'main' }` nunca se cumple y las etapas de despliegue, E2E y rendimiento no se ejecutarían, que fue el defecto de la versión 0.2.

Lo que se asume al usar Jenkins (riesgo R04): Jenkins corre en Docker en la computadora de Lujan, así que GitHub no puede enviarle webhooks; el Multibranch escanea el repositorio cada minuto, lo que cumple el disparo en 3 minutos o menos. Si esa computadora está apagada, ningún PR se puede fusionar, porque la rama main exige el check de Jenkins. **Excepción:** los respaldos (`backup.yml`) corren en GitHub Actions, porque deben ejecutarse cada día a las 03:00 aunque la computadora esté apagada; no es parte del CI.

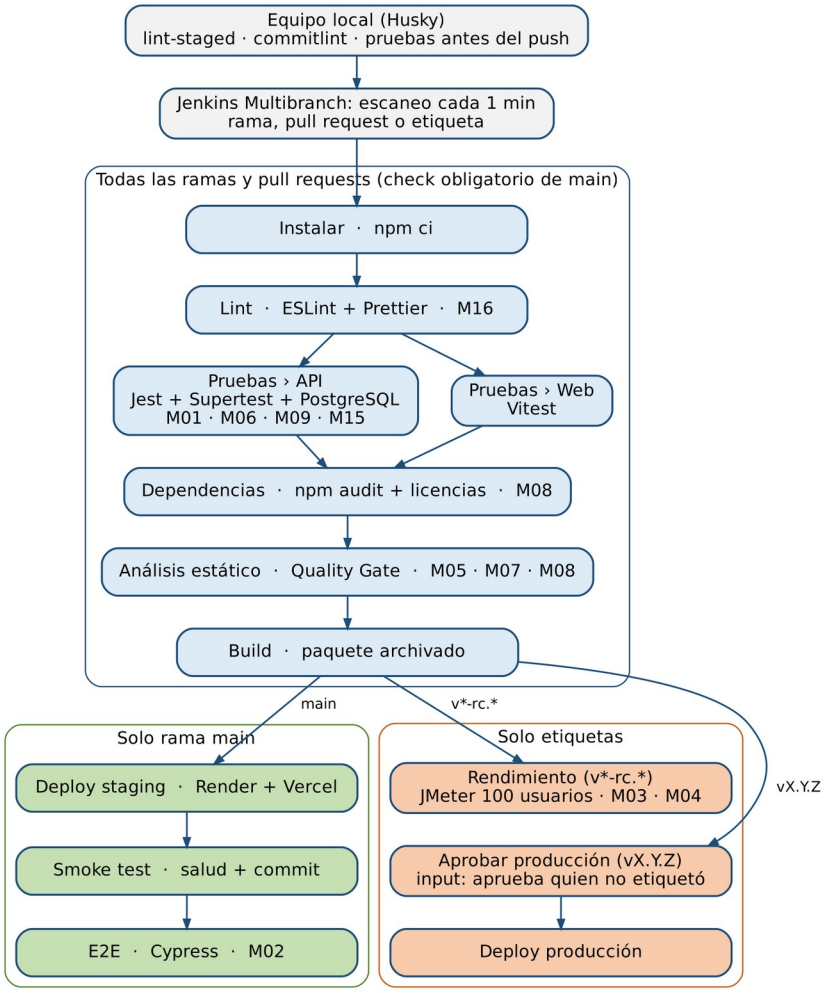


Figura 3. Etapas del Jenkinsfile y métricas que hace cumplir cada una

Por qué SonarQube Cloud. La edición Community de SonarQube no analiza pull requests ni ramas; como M05, M07 y M08 se miden "en cada PR", se usa SonarQube Cloud, gratuito para repositorios públicos. El scanner corre con `-Dsonar.qualitygate.wait=true`, de modo que la etapa espera el resultado del Quality Gate sin necesitar un webhook hacia el Jenkins local. El plan gratuito no permite asignar un Quality Gate propio al proyecto, así que se usa el integrado "Sonar way": sus condiciones sobre el código nuevo incluyen las tres del plan con los mismos umbrales (cobertura menor que 80 %, duplicación mayor que 3 % y seguridad peor que A) y además exigen confiabilidad y mantenibilidad A y el 100 % de los security hotspots revisados. Ese plan, además, solo analiza la rama principal del proyecto, que en SonarQube Cloud es develop: el análisis corre en cada build de develop, donde se trabaja (ADR-0008), y el PR de release a main se apoya en el Quality Gate del último commit de develop.

Técnicas: TDD obligatorio en los cinco módulos críticos; BDD ligero, porque los criterios Dado/Cuando/Entonces se convierten en nombres de pruebas; revisión de cada commit de develop y del PR de release a main (Sección 6, ADR-0008); análisis estático en cada build; integración continua con despliegue continuo a staging; pair programming de una hora por semana sobre los módulos críticos para transferir conocimiento (mitigación de R01). Marco de trabajo: Scrum adaptado a sprints de una semana (planificación el día 1, revisión y retrospectiva el día 5).

10. Control de medios

Esta sección define dónde vive cada artefacto, cómo entra un cambio, cómo se versiona, cómo se protege y cómo se recupera. Incluye la gestión de cambios de las historias de usuario.

10.1 Repositorio

```
calisfacu/                    (monorepo, workspaces de npm)
├─ api/                       NestJS: src/modules/<dominio>, src/database/, test/
├─ web/                       React + Vite
├─ e2e/                       Cypress (*.cy.ts)
├─ perf/                      planes de JMeter (*.jmx) y datos de carga
├─ docs/                      sqap/ srs/ sad/ adr/ backlog/ actas/ evidencias/
│                             manuales/ capacitacion/
├─ .github/                   ISSUE_TEMPLATE/ dependabot.yml pull_request_template.md
│                             workflows/backup.yml (solo respaldos)
├─ jenkins/                   Dockerfile, docker-compose.yml, plugins.txt
├─ Jenkinsfile                pipeline de CI/CD (Multibranch)
├─ .husky/                    pre-commit, commit-msg, pre-push
├─ docker-compose.yml         PostgreSQL 16 y Redis para desarrollo
└─ CHANGELOG.md · README.md · .env.example
```

10.2 Estrategia de ramas, pull requests y merge

Se usa una variante de **Git Flow** (ADR-0008): `main` contiene solo versiones liberadas (producción) y `develop` es la rama de desarrollo e integración (staging), donde se trabaja directamente. **Justificación:** separa lo que está en integración de lo que usa el cliente, y cada release pasa por un PR `develop → main` que revisa el otro integrante. Con dos personas y sprints de una semana, un PR por tarea frenaba la integración; la calidad de develop la cuidan los hooks de Husky, el build de Jenkins de cada push y la revisión de cada commit. No se usan ramas `release/*`: el release candidate es una etiqueta sobre `develop`.

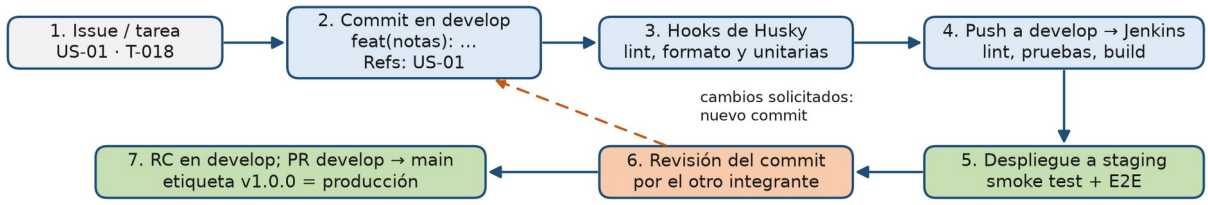


Figura 4. Recorrido de un cambio: de la tarea a la versión

Tabla 21. Reglas de ramas y pull requests

Regla	Configuración
Ramas permanentes	`main` (producción, solo versiones etiquetadas `vX.Y.Z`) y `develop` (integración, se despliega a staging en cada merge).
Trabajo diario	Commits directos en develop (sin ramas feature/ ni fix/), cada uno con su referencia a la historia o al defecto; antes del push corren lint, formato y todas las pruebas unitarias.
Hotfix	`hotfix/CAL-NNN-...` sale de `main`, vuelve a `main` con su etiqueta `vX.Y.Z+1` y se fusiona también a `develop` en el mismo día.

Regla	Configuración
Release	PR `develop` → `main` aprobado por el integrante que no lo abrió; después, la etiqueta `vX.Y.Z` sobre `main` dispara la aprobación y el despliegue a producción.
Protección de main y develop (ruleset)	main: pull request obligatorio con 1 aprobación; descartar aprobaciones si llegan commits nuevos; check obligatorio de Jenkins (continuous-integration/jenkins/pr-merge); rama al día con la destino; sin excepción para administradores. develop: sin PR obligatorio. Ambas: sin force push ni borrado.
Pull request de release	Plantilla del Anexo D: historia o defecto, qué cambia, cómo se probó, checklist de la DoD. Tamaño objetivo: ≤ 400 líneas cambiadas; si es mayor, se divide.
Revisor	Asignado según la Sección 6: Galvan para módulos críticos y API; Lujan para interfaz, CI y scripts de prueba. Revisa los commits de develop en 1 día hábil.
Merge	De develop a main: merge commit, para conservar la historia de la versión. Hotfix: PR a main y después merge de main en develop el mismo día.

10.3 Mensajes de commit

Formato **Conventional Commits 1.0.0**, verificado por commitlint en el hook `commit-msg`. Cada commit referencia la historia o el defecto:

```
<tipo>(<ámbito>): <resumen en imperativo, minúsculas, sin punto final>

<cuerpo opcional: qué y por qué>

Refs: US-01            (o Closes #23 para cerrar el issue)

Tipos:   feat, fix, test, refactor, docs, style, perf, build, ci, chore, revert
Ámbitos: auth, academico, notas, calculo, auditoria, asistencia, importacion,
         boletin, panel, centralizacion, notificaciones, web, e2e, perf, ci, deps, sqap

Ejemplos:
feat(calculo): calcular la nota trimestral según la RM 001/2026
test(calculo): agregar los casos frontera 50,5 y 50,4 de la tabla firmada
fix(boletin): generar en cola los boletines de cursos de más de 50 estudiantes
docs(sqap): actualizar el umbral de M03 en las Secciones 5 y 7
```

Un cambio que rompe compatibilidad (por ejemplo, un contrato de API) lleva `!` después del tipo y la nota `BREAKING CHANGE:` en el cuerpo.

10.4 Versiones y releases

- **Versionado:** Semantic Versioning 2.0.0. Release candidates `v1.0.0-rc.1` (fin de la semana 3) y `v1.0.0-rc.2` (semana 4, día 3); versión final `v1.0.0` (semana 4, día 5).
- Una etiqueta `vX.Y.Z-rc.N` sobre `develop` ejecuta la etapa Rendimiento (M03, M04) sobre ese commit; la etiqueta `vX.Y.Z` sobre `main` ejecuta las etapas de aprobación y despliegue a producción. Las etiquetas se crean sobre commits que ya tienen su build en verde.

- **Producción** espera una aprobación manual en la etapa `input` del `Jenkinsfile`; aprueba el integrante que **no** creó la etiqueta (RACI: aprobar un release). La consola del build registra quién aprobó, y la auditoría verifica que no sea quien etiquetó.
- **CHANGELOG.md** se actualiza en el PR de release, agrupado por tipo de commit.
- **Reversión:** `git revert` del commit en develop (o en un hotfix hacia main); el pipeline vuelve a desplegar. Nunca se reescribe el historial de `develop` ni de `main`.

10.5 Secretos y control de acceso

- Los secretos (URL de bases de datos, `JWT_SECRET`, Deploy Hooks de Render, token de SonarQube, frase de cifrado de respaldos, clave del proveedor de correo) viven solo en **Jenkins Credentials** (tipo *secret text*, usados con `withCredentials`), en el panel de cada plataforma y, para los respaldos, en GitHub Secrets. El repositorio tiene `.env.example` sin valores; `.env` está en `.gitignore`.
- Jenkins: solo los dos integrantes tienen cuenta (seguridad basada en matriz); el token de GitHub de Jenkins tiene únicamente permisos de lectura del repositorio y de escritura de estados de commit; el volumen `jenkins_home` se respalda junto con el bundle del repositorio cada día 5.
- Acceso: ambos integrantes con rol *Maintain* en GitHub y autenticación en dos pasos obligatoria; el propietario del repositorio es Lujan. Producción: solo los dos integrantes; el cliente accede únicamente a la aplicación.
- Si un secreto se expone, se rota en menos de 1 hora hábil y se registra como defecto S1 (riesgo R12).

10.6 Respaldos

Tabla 22. Política de respaldos

Qué	Frecuencia	Retención	Responsable	Prueba de restauración
Base de datos de producción (pg_dump cifrado con AES-256)	Diaria, 03:00 (hora de Bolivia), `backup.yml`	30 días como artefacto del workflow	Configura Galvan; verifica Lujan	Job `restaurar` cada día 5: restaura en un PostgreSQL vacío y compara conteos de tablas (RTO ≤ 2 h, M14)
Base de datos de staging	No se respalda: se recrea con el seed	—	—	—
Repositorio	GitHub es la copia principal; `git bundle` completo cada día 5	4 bundles en el almacenamiento del Líder	Lujan	Clonar desde el bundle en la auditoría final
Documentos	Versionados en `docs/`	Toda la vida del proyecto	Galvan	Incluidos en el bundle

Como el repositorio es público, los artefactos del workflow pueden ser descargados por cualquier usuario de GitHub; por eso el respaldo se cifra antes de subirlo y la frase de cifrado está solo en GitHub Secrets. La base gratuita de Render vence a los 30 días y no tiene respaldos propios; el respaldo diario y la restauración probada cubren ese riesgo (R10).

10.7 Control de cambios de las historias de usuario

Un cambio de historia (nueva, modificada, dividida o descartada) o de un criterio de aceptación sigue este procedimiento:

1. Cualquier integrante o el cliente abre un issue con el formulario **Solicitud de cambio** (Anexo D): historia afectada, cambio pedido, motivo y quién lo pide.
2. El Líder evalúa el impacto en 1 día hábil: puntos de historia, sprint afectado, criterios y pruebas a cambiar, y si toca un RNF o un umbral del SQAP.
3. Decisión: el Líder aprueba o rechaza con el cliente (RACI). Si el cambio está dentro del sprint en curso, solo entra si no compromete el objetivo del sprint; si no, va al siguiente. Si cambia el alcance aprobado, también lo aprueba el docente.
4. Si se aprueba, en un mismo PR se actualizan la pila de producto (`backlog_v2_4sprints.xlsx` y GitHub Projects), los criterios de aceptación y las pruebas; si cambia un RNF o un umbral, también el SQAP según la Sección 16.
5. Los identificadores nunca se reutilizan: una historia descartada queda con estado Descartada y una historia dividida genera identificadores nuevos (por ejemplo, US-21 y US-22) con referencia a la original.
6. El cambio se comunica en la revisión del sprint y queda en el acta.

11. Control de proveedores

CalisFacu depende de librerías de npm y de servicios en la nube. Ninguna dependencia entra al proyecto sin cumplir los criterios de la Tabla siguiente, y cada una tiene una fila en la hoja Componentes de terceros de `matrices-sqap.xlsx`.

Tabla 23. Criterios de selección de dependencias

Criterio	Regla para aceptar una dependencia
Licencia	Permisiva y compatible: MIT, Apache-2.0, BSD-2-Clause, BSD-3-Clause, ISC o 0BSD. La etapa Dependencias del `Jenkinsfile` la verifica con `license-checker --onlyAllow`.
Mantenimiento activo	Al menos un release en los últimos 12 meses y repositorio no archivado.
Comunidad	Uso amplio (más de 10 000 descargas semanales en npm) o recomendación en la documentación oficial de NestJS o React.
Seguridad	0 vulnerabilidades altas o críticas conocidas al agregarla (M08).
Necesidad	No existe ya una dependencia del proyecto que cubra la misma función.

Procedimiento para agregar una dependencia: PR con la justificación en la descripción y la fila nueva en la hoja Componentes de terceros; lo aprueba el Arquitecto (Lujan) o, si lo propone Lujan, el Responsable SQA. **Monitoreo:** Dependabot revisa npm y GitHub Actions una vez por semana y abre PR de actualización; `npm audit --audit-level=high` corre en cada PR; el primer lunes de cada mes Lujan revisa la hoja y actualiza la fecha de última revisión. Una alerta alta o crítica se trata como defecto S2.

Tabla 24. Dependencias directas principales (las versiones exactas están en package-lock.json)

Componente	Licencia	Uso en CalisFacu
NestJS (`@nestjs/*`)	MIT	Framework de la API

Componente	Licencia	Uso en CalisFacu
TypeORM (`typeorm`, `@nestjs/typeorm`) + pg	MIT	ORM, entidades y migraciones de PostgreSQL
Passport + `@nestjs/jwt`	MIT	Autenticación (US-17)
argon2	MIT	Hash de contraseñas
class-validator / class-transformer	MIT	Validación de DTO
decimal.js	MIT	Aritmética exacta del cálculo de notas (RNF-01)
pdfkit	MIT	Boletín en PDF (US-07); se usa directo, sin pdfmake
BullMQ + ioredis	MIT	Cola de boletines (RNF-03, R05)
exceljs	MIT	Importación (US-05) y exportación a Excel (US-09)
React, Vite	MIT	Aplicación web
Jest, Supertest, Vitest, Cypress	MIT	Pruebas
Apache JMeter	Apache-2.0	Pruebas de carga

Tabla 25. Proveedores de servicios

Servicio	Uso	Riesgo	Plan alternativo
GitHub (repositorio, Issues, Actions solo para respaldos)	Código, defectos, respaldos	Caída temporal del servicio	Bundle del repositorio (Sección 10.6); los respaldos se ejecutan a mano con el mismo script.
Jenkins (autoalojado en Docker)	CI/CD	La computadora que lo aloja está apagada o falla	Reinstalación desde `jenkins/` probada en la semana 1; restauración de `jenkins_home` (R04).
Render (API, PostgreSQL, Redis)	Staging y producción	Plan gratuito: base que vence a los 30 días, sin respaldos y con suspensión por inactividad	Respaldo diario cifrado; migración a otro PostgreSQL administrado restaurando el último respaldo (R10).
Render (sitio estático)	Aplicación web	Límites del plan gratuito	Build estático desplegable en cualquier hosting estático.
SonarQube Cloud	Quality Gate	Servicio no disponible	El PR espera; ESLint y la cobertura de Jest siguen bloqueando.
Proveedor de correo con API HTTP	Avisos de US-08 y US-12	Render gratuito bloquea los puertos SMTP 25, 465 y 587	Se usa la API HTTP del proveedor; el aviso dentro del sistema funciona aunque falle el correo.

12. Registros: recolección, mantenimiento y retención

Tabla 26. Catálogo de evidencias

Evidencia	Ubicación	Formato	Retención	Responsable
Actas de revisión, inspección, auditoría, aceptación y postmortem	`docs/actas/` en el repositorio	PDF firmado o Markdown con aprobación en el PR	Toda la vida del proyecto	Galvan (SQA)

Evidencia	Ubicación	Formato	Retención	Responsable
Informes de Jest y cobertura (M01, M05, M06, M09, M15)	Test Result y artefactos de cada build de Jenkins; los de cada RC, también en `docs/evidencias/<versión>/`	JUnit XML, lcov, HTML	Últimos 30 builds por rama (`buildDiscarder`); los de RC, toda la gestión	Galvan (SQA)
Evidencias de Cypress (M02)	Artefactos del build de main; RC en `docs/evidencias/`	Videos, capturas, JUnit XML	Últimos 30 builds; RC, toda la gestión	Galvan (Tester)
Dashboards de JMeter (M03, M04)	Artefactos `perf/**` del build de cada etiqueta RC	HTML + `statistics.json`	Toda la gestión (copiados a `docs/evidencias/`)	Galvan (Tester)
Registros del pipeline	Console Output de Jenkins	Texto	Últimos 30 builds por rama	Galvan (SQA)
Análisis de SonarQube Cloud (M05, M07, M08)	Proyecto CalisFacu en SonarQube Cloud	Web	Toda la gestión	Galvan (SQA)
Pruebas de usabilidad (M11, M12)	`docs/evidencias/usabilidad/`	Planilla y cuestionarios SUS anonimizados	Toda la gestión	Galvan (Tester)
Disponibilidad (M13)	Panel de UptimeRobot; exportación mensual en `docs/evidencias/`	CSV	Toda la gestión	Galvan (SQA)
Respaldos y pruebas de restauración (M14)	Artefactos de `backup.yml` (GitHub Actions); resultado del job `restaurar`	`.dump.gpg`; log	30 días; resultados, toda la gestión	Lujan (Dev)
Defectos y solicitudes de cambio	GitHub Issues	Issue con formulario	Toda la vida del proyecto	Lujan (Líder)
Revisiones de código	Pull requests de GitHub	Comentarios y aprobaciones	Toda la vida del proyecto	Lujan (Líder)

El número 30 de la retención de builds está en el plan y en `options { buildDiscarder(logRotator(numToKeepStr: '30')) }` del `Jenkinsfile`; el 30 de los respaldos (días), en `retention-days: 30` de `backup.yml`. Como Jenkins descarta builds viejos, las evidencias de cada release candidate se copian a `docs/evidencias/` en la auditoría de cierre.

13. Capacitación

El cambio a NestJS y PostgreSQL deja brechas que deben cerrarse en la semana 1 para no frenar los sprints siguientes (riesgo R08).

Tabla 27. Competencias y brechas por rol

Rol	Competencia requerida	Nivel actual	Brecha
Dev backend (Lujan)	NestJS (módulos, guards, pipes), TypeORM y migraciones, Jest con TDD, BullMQ	Laravel y PHPUnit (Prácticas 1 y 2 con JUnit)	Alta en NestJS y TypeORM
Frontend / Tester (Galvan)	React + TypeScript, Cypress con TypeScript, JMeter, Vitest	Cypress y JMeter (Prácticas 4 y 5)	Media en React con TypeScript

Rol	Competencia requerida	Nivel actual	Brecha
Responsable SQA (Galvan)	Jenkins Multibranch, credenciales, rulesets, SonarQube Cloud, Husky, IEEE 730 e IEEE 1028	Jenkins y SonarQube (Prácticas 3 y 6)	Media en Multibranch
Todos	Reglamento de evaluación del Ministerio (RM 001/2026) y glosario del dominio	Ninguna	Alta

Tabla 28. Plan de capacitación

Sesión	Cuándo	Quién la da	Duración	Evidencia
Reglamento de evaluación y tabla de casos	Semana 1, día 1	Profesor del cliente	1 h	Acta y tabla de casos firmada
NestJS + TypeORM: módulo de ejemplo con TDD	Semana 1, día 1	Lujan (preparada con la documentación oficial)	2 h	Repositorio de ejemplo en `docs/capacitacion/`
Pipeline: Jenkins Multibranch, credenciales, Husky y rulesets	Semana 1, día 2	Galvan	1 h	Primer PR con el check de Jenkins en verde
React + TypeScript y Cypress con TypeScript	Semana 1, día 2	Galvan	1 h	Primer escenario E2E en verde
Pair programming en módulos críticos	Cada semana, día 3	Lujan con Galvan	1 h	Nota en el issue de la tarea

Criterio de cierre de la brecha: al terminar la semana 1, cada integrante fusionó al menos un PR en el área de la brecha que pasó todos los checks y la revisión del otro.

14. Gestión de riesgos

Escala de probabilidad (P) e impacto (I) de 1 (bajo) a 3 (alto); exposición = P × I; nivel: 1–2 bajo, 3–4 medio, 6–9 alto. Todo riesgo alto tiene una mitigación que se puede poner como tarea en el tablero, con responsable y momento. El disparador indica cuándo se activa el plan de contingencia. Los riesgos se revisan en cada retrospectiva (día 5).

Tabla 29. Matriz de riesgos de calidad

ID	Riesgo	P	I	Exp.	Mitigación accionable	Resp.	Disparador	Contingencia
R01	Abandono o ausencia prolongada de un integrante (equipo de dos)	3	3	9 Alto	Cada historia se documenta en el issue antes de cerrarse; todo PR lo revisa el otro; pair programming semanal en módulos críticos.	Lujan	Un integrante sin commits ni asistencia 3 días hábiles	El otro asume; se renegocia el alcance con el docente priorizando US-01 y US-02.
R02	Reglas de evaluación mal interpretadas (dimensiones, redondeo, nota anual)	2	3	6 Alto	Tabla de casos según la RM 001/2026 firmada por el cliente el día 1 de la semana 1; dimensiones configurables por gestión (US-18).	Lujan	Un caso de prueba sin resultado acordado	Sesión con el cliente en 24 h; la regla se congela hasta tener acta.
R03	Deuda técnica por presión de cronograma	2	3	6 Alto	Quality Gate bloqueante (M05, M07); 10 % de las horas de cada sprint para refactorizar.	Galvan	Quality Gate en rojo en 2 PR seguidos	Medio día del sprint siguiente solo para estabilizar.

ID	Riesgo	P	I	Exp.	Mitigación accionable	Resp.	Disparador	Contingencia
R04	Jenkins no disponible (corre en la computadora de Lujan) y bloquea los merges a main	2	3	6 Alto	Jenkins en Docker con `jenkins/Dockerfile` y `docker-compose.yml` versionados; reinstalación desde cero probada en la semana 1, día 2; respaldo de `jenkins_home` cada día 5; computadora encendida en horario hábil.	Galvan	Jenkins caído más de 2 horas hábiles	Reinstalar en la computadora de Galvan desde `jenkins/` y restaurar `jenkins_home`; mientras tanto, nada se fusiona a main.
R05	Lentitud o fallas al generar boletines en el cierre de trimestre	3	3	9 Alto	Cola BullMQ para cursos de más de 50 estudiantes (tarea de la semana 3); prueba de carga en cada RC (M03, M04).	Lujan	p95 > 2000 ms o tasa de error ≥ 1 % en la etapa Rendimiento	Generación diferida con aviso al profesor cuando el PDF está listo.
R06	Vulnerabilidad crítica en una dependencia	2	3	6 Alto	Dependabot semanal y `npm audit` en cada PR (M08); revisión mensual de la hoja de componentes.	Lujan	Alerta alta o crítica	Fijar la última versión segura o reemplazar la librería en 48 h.
R07	Datos reales de estudiantes (menores) en ambientes de prueba o en el repositorio	1	3	3 Medio	Seeds sintéticos con faker; prohibido copiar la base de producción; revisión en cada PR.	Galvan	Aparece un nombre o documento real fuera de producción	Borrar los datos, rotar credenciales, reescribir el artefacto e informar al docente.
R08	Curva de aprendizaje de NestJS, TypeORM y React con TypeScript	3	2	6 Alto	Sesiones de capacitación de la semana 1 (Sección 13); módulo de ejemplo con TDD antes de US-01.	Lujan	Una tarea supera el doble de sus horas estimadas	Pair programming inmediato y reestimación en la planificación siguiente.
R09	Velocidad insuficiente para 4 sprints de una semana (la semana 1 está sobrecargada)	3	3	9 Alto	Medir la velocidad el día 5 de la semana 1; regla de recorte acordada: salen primero US-08 y US-12 (prioridad baja).	Lujan	Menos del 80 % de los puntos de la semana 1 terminados	Aplicar el recorte y comunicarlo al cliente y al docente en la revisión del sprint.
R10	Límites del plan gratuito de Render (base que vence a los 30 días, sin respaldos, suspensión)	2	2	4 Medio	Respaldo diario cifrado y restauración probada (M14); monitor de disponibilidad (M13).	Galvan	Aviso de vencimiento o restauración fallida	Restaurar el último respaldo en otro PostgreSQL administrado y actualizar el secreto.
R11	Cambio de la normativa de evaluación durante la gestión	1	3	3 Medio	Puntajes de dimensiones como parámetros por gestión, no en el código (US-18).	Lujan	Nueva resolución del Ministerio	Configurar la nueva gestión y agregar sus casos a la tabla firmada.
R12	Exposición de un secreto (token o URL de base de datos)	1	3	3 Medio	Secretos solo en Jenkins Credentials (y GitHub Secrets para respaldos); `withCredentials` enmascara su valor en la consola; escaneo de secretos de GitHub activado.	Galvan	Alerta de escaneo de secretos o secreto visible en un log	Rotar en menos de 1 hora hábil y registrar un defecto S1.
R13	Cliente no disponible para inspección, demo o aceptación	2	3	6 Alto	Calendario de revisiones acordado con el cliente el día 1 de la semana 1, con un suplente nombrado.	Lujan	El cliente cancela una revisión	Revisión asíncrona sobre staging con un plazo de 24 h; si no hay respuesta, se escala al docente.
R14	Cambios de historias tarde en el sprint	2	2	4 Medio	Procedimiento de cambio de la Sección 10.7: solo entra si no compromete el objetivo del sprint.	Lujan	Solicitud de cambio después del día 3	El cambio pasa al sprint siguiente o a la Release 2.

15. Glosario

Términos del dominio (primera parte de la tabla) y técnicos (segunda parte). El código, la interfaz y las pruebas usan estos mismos términos.

Tabla 30. Glosario

Término	Definición
Actividad	Tarea evaluada (examen, práctica, exposición) dentro de una dimensión; se califica de 1 a 100.
Año de escolaridad	Grado que cursa el estudiante dentro de su nivel (por ejemplo, 3.° de Secundaria).
Autoevaluación	Dimensión en la que el estudiante valora su propio desempeño; 5 puntos según la RM 001/2026.
Boletín	Documento PDF con las notas trimestrales y anuales y la asistencia de un estudiante (US-07).
Centralizador	Planilla consolidada de un curso con las notas de todas sus materias, con el formato oficial (US-09).
Código SIE	Código de la unidad educativa en el Sistema de Información Educativa del Ministerio de Educación.
Curso	Combinación de año de escolaridad y paralelo en una gestión (por ejemplo, 3.° A de Secundaria, 2026).
Dimensión	Componente de la evaluación con puntaje máximo fijado por el Ministerio: Ser, Saber, Hacer y Autoevaluación.
Gestión	Año escolar (por ejemplo, gestión 2026), dividido en tres trimestres.
Nivel	Etapa del Subsistema de Educación Regular: Inicial, Primaria o Secundaria.
Nota anual	Nota de una materia en la gestión, promedio de las tres notas trimestrales redondeado al entero, 0,5 hacia arriba (decisión D3).
Nota mínima de aprobación	51 puntos en la escala de 1 a 100.
Nota trimestral	Suma de los puntos de las cuatro dimensiones, redondeada al entero (US-02).
Paralelo	Grupo dentro de un año de escolaridad (A, B, C…).
Trimestre	Periodo de evaluación; la gestión tiene tres.
Unidad educativa	Institución escolar cliente del sistema.
ADR	Architecture Decision Record: nota breve que registra una decisión de arquitectura, sus alternativas y consecuencias.
BDD	Desarrollo guiado por comportamiento: criterios Dado/Cuando/Entonces que se convierten en pruebas.
Conventional Commits	Formato de mensajes de commit `tipo(ámbito): resumen` (Sección 10.3).
Definition of Done (DoD)	Condiciones para considerar una historia Hecha (hoja DoR y DoD del backlog).
Definition of Ready (DoR)	Condiciones para que una historia entre a un sprint.
Feature test / e2e-spec	Prueba que ejercita una ruta HTTP completa de la API contra PostgreSQL, con Supertest.
Guard	Componente de NestJS que decide si una petición puede llegar al controlador (autorización).
Migración	Cambio versionado del esquema de la base de datos, generado y aplicado con la CLI de TypeORM (`api/src/database/migrations/`).

Término	Definición
Módulo crítico	Módulo cuya falla afecta notas oficiales o el acceso a ellas (Tabla 4).
p95	Valor por debajo del cual está el 95 % de los tiempos de respuesta medidos.
Build sano	Build que cumple los indicadores de las Prácticas 6 y 8 de la Sección 6.2, no solo el color verde.
Prioridad	Con qué urgencia se corrige un defecto (P1 a P3). La fija el Líder.
Quality Gate	Conjunto de condiciones de SonarQube Cloud que un cambio debe cumplir para fusionarse.
RACI	Matriz que asigna, por actividad, quién ejecuta (R), rinde cuentas (A), es consultado (C) o informado (I).
Release candidate (RC)	Versión candidata etiquetada `v*-rc.*`, sobre la que se corren E2E y rendimiento.
RPO / RTO	Pérdida máxima de datos tolerada / tiempo máximo de restauración tras una falla.
Secreto	Dato de acceso (token, contraseña, URL con credenciales) que vive solo en Jenkins Credentials, GitHub Secrets o el panel de la plataforma.
SemVer	Versionado semántico MAYOR.MENOR.PARCHE.
Severidad	Cuánto daño causa un defecto (S1 a S4). La fija el Tester o el Responsable SQA.
SLA	Tiempos máximos de respuesta y resolución de un defecto según su severidad.
Smoke test	Verificación breve tras un despliegue: salud del servicio y commit desplegado.
Staging	Ambiente con la configuración de producción y datos sintéticos.
SUS	System Usability Scale: cuestionario de 10 ítems con resultado de 0 a 100.
TDD	Desarrollo guiado por pruebas: se escribe la prueba antes que el código.

16. Procedimiento de cambio e historia del SQAP

Cualquier integrante propone un cambio con un pull request que modifica `docs/sqap/SQAP-CalisFacu.docx` y su PDF. Como el .docx no se puede comparar línea por línea en GitHub, la descripción del PR lista las secciones y tablas cambiadas con el texto antes y después. Lo aprueba el Responsable SQA; si el cambio modifica un umbral o el alcance, también lo aprueba el Líder, y en el mismo PR se actualizan `matrices-sqap.xlsx` y la configuración de la herramienta. Si el cambio reduce el alcance aprobado, lo aprueba además el docente. Cada cambio es una fila del historial: una corrección, una versión.

Tabla 31. Autoría por sección

Secciones	Autor	Revisor
1, 2, 3, 4, 10, 13, 15 y 16	Lujan	Galvan
5, 6, 7, 8, 9, 11, 12 y 14; Anexos	Galvan	Lujan

Tabla 32. Historial de versiones del SQAP

Versión	Fecha	Cambio	Autor	Revisor
0.1	04/10/2026	Versión inicial: Secciones 1 a 8.	Lujan	Galvan

Versión	Fecha	Cambio	Autor	Revisor
0.2	04/10/2026	Corrección tras la revisión contra los siete indicadores: métricas M01 a M09 iguales en el plan, en matrices-sqap.xlsx y en la configuración; RACI con independencia; SLA en horario hábil; diagrama de estados; Secciones 9, 12, 14 y 15; Anexo de RNF.	Galvan	Lujan
0.3	12/10/2026	Sección 1: alcance trazable al backlog v2.0 (roles Profesor, Secretaria, Padre de familia y Administrador; 16 historias en 4 sprints de una semana); módulo de pagos fuera del alcance; condiciones que elevarían el nivel de integridad a 3.	Lujan	Galvan
0.4	12/10/2026	Secciones 1, 5, 7, 9 y 11: cambio de stack de Laravel + MySQL a NestJS + PostgreSQL + React. PSR-12 pasa a typescript-eslint + Prettier (M16); PHPUnit + PCOV a Jest + Supertest; composer audit a npm audit; DomPDF a pdfmake; Laravel Queue a BullMQ.	Galvan	Lujan
0.5	12/10/2026	Secciones 9 y 10: Jenkins se configura como Multibranch Pipeline con builds de PR y de etiquetas. Corrige que `when { branch 'release/*' }` nunca se cumplía en un job Pipeline simple, por lo que E2E y rendimiento no se ejecutaban. Se agregan despliegue a staging, smoke test, aprobación manual de producción y la imagen `jenkins/Dockerfile`.	Galvan	Lujan
0.6	12/10/2026	Secciones 5 y 9: SonarQube Community se reemplaza por SonarQube Cloud, porque Community no analiza pull requests y M05, M07 y M08 se miden en cada PR.	Galvan	Lujan
0.7	12/10/2026	Anexo A: RNF-02 y RNF-05 pasan a tener métrica y valor (incumplían el criterio de adecuación de la Sección 4); RNF-01 sigue la RM 001/2026; se agregan RNF-06 (usabilidad) y RNF-07 (fiabilidad).	Lujan	Galvan
0.8	12/10/2026	Sección 5: seis características con dos o más métricas (M11 a M16); M04 exige la tasa de error también en el boletín; el estándar de código se hace cumplir con una herramienta (M16).	Galvan	Lujan
0.9	12/10/2026	Sección 8: si Galvan resuelve un defecto, verifica y cierra Lujan; escala de prioridad P1 a P3.	Galvan	Lujan
0.10	12/10/2026	Secciones 4 y 6: inspección Fagan del SRS (además del SAD) con roles, etapas y métricas; revisión de diseño detallado; manual técnico; calendario por día de sprint.	Galvan	Lujan
0.11	12/10/2026	Secciones 10, 11 y 13 completas (antes "Pendiente"); 14 riesgos con disparador y contingencia; glosario de 40 términos.	Lujan	Galvan
0.12	12/10/2026	Sección 10.2: GitHub Flow se reemplaza por Git Flow (ramas develop y main, hotfix desde main), como enseñó el docente; develop despliega a staging y main a producción; ambas ramas protegidas.	Lujan	Galvan
0.13	12/10/2026	Secciones 1, 4, 5, 7, 9, 11, 13, 14 y 15: Prisma se reemplaza por TypeORM por decisión del equipo: es el ORM de la documentación oficial de NestJS, está escrito en JavaScript y no depende de binarios propios que se descargan aparte; las pruebas e2e se integran en `test:cov` para que M06 mida la cobertura combinada.	Galvan	Lujan
1.0	12/10/2026	Consolidado: portada, control del documento, índice, listas de tablas y figuras, agrupación según IEEE 730-2014 y Anexos A a F.	Lujan	Galvan
1.1	13/10/2026	Secciones 3, 6, 8, 9 y 10.2: se trabaja directo en develop y la revisión de código pasa a ser revisión de commits; main sigue protegida por el PR de release (ADR-0008). Secciones 1, 7, 10, 11 y Anexo C: la web se despliega en Render y no en Vercel, y el despliegue continuo a staging funciona desde el Sprint 1 (ADR-0009).	Lujan	Galvan

Versión	Fecha	Cambio	Autor	Revisor
1.2	30/10/2026	Revisión de consistencia antes del cierre del Sprint 3. Sección 1, Anexo B y Anexo C: familias (vínculo padre–estudiante, US-20 y consulta del padre de US-11) pasa a ser módulo crítico y entra al coverageThreshold de M06. Sección 11: pdfkit reemplaza a pdfmake y se agrega ioredis. Sección 15 y Anexo F: los pendientes de reglas se cierran con las decisiones D1 a D20. matrices-sqap.xlsx, hoja Métricas: la columna "Dónde se configura" nombra las etapas del Jenkinsfile y no workflows de GitHub Actions que no existen, y M02 se mide en cada build de develop, como dice la Tabla 9. Tabla 1, Sección 3 y Anexo F: cliente piloto, la Unidad Educativa Nazareno, representada por la directora Fabiana Parrado y la profesora Fatima Carreño. Sección 1 y Anexo F: el docente aprobó US-17 a US-20 y dejar los pagos fuera. Sección 9 y 12: el monitor de M13 es UptimeRobot y el equipo se comunica por WhatsApp y Discord. Secciones 5 y 9 y Anexo C: SonarQube Cloud se conectó el 30/10/2026 (T-030); el Quality Gate es "Sonar way", porque el plan gratuito no permite uno propio, y contiene las condiciones de M05, M07 y M08 con los mismos umbrales; como el plan gratuito solo analiza la rama principal, esta es develop y M05, M07 y M08 se miden en cada build de develop.	Lujan	Galvan

Rige desde la versión 1.0, antes del inicio de la semana 1. Las versiones 0.2 a 0.13 se registran por separado para que cada corrección sea identificable, como pide la Práctica 7.

Anexo A — Requerimientos no funcionales (SRS 1.0)

Tabla 33. Requerimientos no funcionales

Código	Requerimiento	Métricas
RNF-01	La nota trimestral se calcula según el reglamento vigente: puntos de cada dimensión = promedio de sus actividades × puntaje máximo / 100, con Ser 10, Saber 45, Hacer 40 y Autoevaluación 5 en 2026, configurables por gestión; nota trimestral = suma de las dimensiones redondeada al entero (0,5 hacia arriba), en la escala de 1 a 100; aprueba con 51. El 100 % de los casos de la tabla firmada por el cliente da el resultado esperado.	M01, M06
RNF-02	Un profesor registra las notas de toda una actividad (hasta 60 estudiantes) en una sola operación: se guardan todas o ninguna. El 100 % de los escenarios E2E de registro, corrección y boletín pasan en staging.	M02
RNF-03	El boletín de un estudiante se genera con p95 ≤ 2000 ms y tasa de error < 1 % con 100 usuarios concurrentes durante 10 minutos; la planilla de notas se registra con tasa de error < 1 % en las mismas condiciones. Los boletines de cursos de más de 50 estudiantes se generan en cola.	M03, M04
RNF-04	Las notas son datos personales de menores y datos oficiales: solo el profesor asignado a la materia las modifica; un padre solo ve a sus hijos vinculados; el 100 % de las rutas protegidas rechazan con 403 al usuario sin permiso; el 100 % de los cambios de nota quedan auditados; 0 vulnerabilidades altas o críticas.	M08, M09, M15
RNF-05	El código puede mantenerlo un equipo nuevo: cobertura ≥ 80 % en código nuevo y ≥ 90 % en módulos críticos; duplicación ≤ 3 %; 0 errores y advertencias de lint y formato.	M05, M06, M07, M16
RNF-06	Un profesor que nunca usó el sistema registra las notas de una actividad sin ayuda: al menos 4 de 5 profesores lo logran y el SUS promedio es ≥ 68.	M11, M12
RNF-07	Producción está disponible al menos el 99,0 % del mes; la base se respalda cada día (RPO ≤ 24 h) y se restaura en 2 horas o menos (RTO ≤ 2 h).	M13, M14

Anexo B — Trazabilidad historia → módulo → pruebas → métricas

Tabla 34. Trazabilidad de las historias del alcance

Historia	Módulo	Pruebas que la verifican	Métricas
US-17 Autenticación y roles	auth (crítico)	Jest: guards y bloqueo; Supertest: `autorizacion.e2e-spec.ts`; Cypress: inicio de sesión por rol	M09, M06
US-18 Gestión académica base	academico; calculo (dimensiones)	Jest: validaciones y dimensiones que suman 100; Supertest: CRUD; Cypress: secretaria	M01, M06
US-01 Registro central de notas	notas (crítico)	Jest (TDD): rango 1 a 100, transacción; Supertest; Cypress: Registrar calificación; JMeter: planilla	M01, M02, M04, M06
US-02 Cálculo automático	calculo (crítico)	Jest (TDD): `calculo.casos.spec.ts` con la tabla firmada	M01, M06
US-04 Corrección auditable	auditoria (crítico)	Jest (TDD); Supertest: `auditoria.e2e-spec.ts`; Cypress	M15, M06, M02
US-03 Historial académico	notas	Supertest: 403 para estudiantes ajenos; Cypress	M09, M02

Historia	Módulo	Pruebas que la verifican	Métricas
US-19 Asistencia	asistencia	Jest: porcentaje y fechas; Cypress	M05
US-05 Importación desde Excel	importacion → notas	Jest: validación por fila; Supertest: auditoría con origen; Cypress	M15, M06
US-10 Control de entrega	centralizacion	Jest; Cypress	M05
US-06 Panel de riesgo	panel	Jest: umbral de riesgo; Cypress	M05
US-07 Boletín en PDF	boletin	Jest: contenido; Supertest: cola; Cypress; JMeter: boletín	M02, M03, M04
US-20 Vinculación padre–estudiante	familias (crítico)	Supertest: 403 al cambiar el ID en la URL; Cypress	M09, M06
US-11 Consulta de boletín	boletin; familias (consulta del padre)	Supertest: boletín no publicado invisible; Cypress	M02, M09, M06
US-09 Centralización	centralizacion	Jest: materias pendientes; Supertest: formato oficial; Cypress	M02
US-08 Alerta de promedio bajo	notificaciones	Jest: sin alertas repetidas; Supertest	M05
US-12 Aviso de boletín	notificaciones	Supertest	M05

Anexo C — Configuración que hace cumplir el plan

Estos archivos son la versión inicial que se crea en la semana 1 (tareas de infraestructura de la pila del sprint). Los comentarios indican la métrica que hace cumplir cada etapa. Los textos entre < > se completan al crear las cuentas.

Tabla 35. Configuración del job Multibranch Pipeline

Opción del job Multibranch "calisfacu"	Valor
Branch Sources	GitHub `lujan-99/calisfacu`, con un token de solo lectura del repositorio más permiso de estados de commit (credencial `github-calisfacu`)
Behaviours	Discover branches (excluir las que también son PR); Discover pull requests from origin (merge con la rama destino); Discover tags
Build strategies	Regular branches; Change requests; Tags (ignorar etiquetas de más de 7 días). Requiere el plugin basic-branch-build-strategies; sin él, las etiquetas no se construyen solas
Build Configuration	by Jenkinsfile, ruta `Jenkinsfile`
Scan Multibranch Pipeline Triggers	Periodically if not otherwise run: 1 minuto (Jenkins es local y no recibe webhooks de GitHub)
Credenciales (secret text)	`sonar-token, render-hook-staging, render-hook-web-staging, render-hook-prod, render-hook-web-prod`
Seguridad	Matrix-based security: solo `lujan` y `galvan`; sin acceso anónimo

C.1 Jenkinsfile

Nota (versión 1.1): la copia de este anexo es la de la versión 1.0. La vigente es el Jenkinsfile de la raíz del repositorio (aprobación independiente, despliegue en Render y SonarQube condicionado a SONAR_HABILITADO).

```
// Jenkinsfile — job Multibranch Pipeline "calisfacu" (ramas, pull requests y etiquetas).
// Flujo de ramas (docs/desarrollo/flujo-git.md): feature/* y fix/* → develop (staging) → main
(producción).
pipeline {
  agent any
  options {
    timestamps()
    timeout(time: 45, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '30'))         // Sección 12: últimos 30 builds
    disableConcurrentBuilds()
  }
  environment {
    PGHOST = 'postgres-ci'                                 // base de CI desechable
    PGUSER = 'calis'
    PGPASSWORD = 'calis'
    DB_NAME = "ci_${env.BUILD_TAG.replaceAll('[^A-Za-z0-9]', '_').toLowerCase()}"
    DATABASE_URL = "postgresql://calis:calis@postgres-ci:5432/${env.DB_NAME}"
    REDIS_URL = 'redis://redis-ci:6379'
  }
  stages {
    stage('Instalar') { steps { sh 'node -v && npm ci' } }
    stage('Lint') {                                        // M16
      steps { sh 'npx eslint . --max-warnings=0 && npx prettier --check .' }
    }
    stage('Pruebas') {
      parallel {
        stage('API') {                                     // M01, M06, M09, M15
          steps {
            sh 'createdb "$DB_NAME"'
            // Unitarias + e2e contra PostgreSQL (migraciones aplicadas en el globalSetup de
Jest).
            // Incluye calculo.casos (M01), autorizacion (M09), auditoria (M15) y
coverageThreshold (M06).
            sh 'npm run -w api test:cov'
          }
          post {
            always {
              junit 'api/reports/*.xml'                    // build sano: informe publicado
              sh 'dropdb --if-exists "$DB_NAME"'
            }
          }
        }
        stage('Web') {
          steps { sh 'npm run -w web test -- --run --coverage' }
          post { always { junit 'web/reports/*.xml' } }
        }
      }
    }
    stage('Dependencias') {                                // M08 y licencias (Sección 11)
      steps {
        sh 'npm audit --audit-level=high'
        sh 'npx license-checker --production --onlyAllow "MIT;Apache-2.0;BSD-2-Clause;BSD-3-
Clause;ISC;0BSD"'
      }
```

```
    }
    stage('Análisis estático') {                          // M05, M07, M08: Quality Gate
"Sonar way"
      when { not { buildingTag() } }
      steps {
        withCredentials([string(credentialsId: 'sonar-token', variable: 'SONAR_TOKEN')]) {
          script {
            def destino = env.CHANGE_ID ?
              "-Dsonar.pullrequest.key=${env.CHANGE_ID} -Dsonar.pullrequest.branch=$
{env.CHANGE_BRANCH} -Dsonar.pullrequest.base=${env.CHANGE_TARGET}" :
              "-Dsonar.branch.name=${env.BRANCH_NAME}"
            sh "sonar-scanner -Dsonar.qualitygate.wait=true ${destino}"
          }
        }
      }
    }
    stage('Build') {
      steps {
        sh 'npm run -w api build && npm run -w web build'
        sh 'tar czf "calisfacu-${GIT_COMMIT}.tgz" api/dist web/dist'
        archiveArtifacts artifacts: 'calisfacu-*.tgz', fingerprint: true   // build sano:
artefacto
      }
    }
    stage('Deploy staging') {
      when { branch 'develop' }                           // nunca antes de que pasen las
pruebas
      steps {
        withCredentials([string(credentialsId: 'render-hook-staging', variable: 'HOOK'),
                         string(credentialsId: 'vercel-token', variable: 'VERCEL_TOKEN'),
                         string(credentialsId: 'vercel-org-id', variable: 'VERCEL_ORG_ID'),
                         string(credentialsId: 'vercel-project-id', variable:
'VERCEL_PROJECT_ID')]) {
          sh 'curl -fsS -X POST "$HOOK&ref=$GIT_COMMIT"'
          sh 'cd web && npx vercel deploy --yes --token="$VERCEL_TOKEN"'
        }
      }
    }
    stage('Smoke test') {
      when { branch 'develop' }
      steps { sh './scripts/smoke.sh "$STAGING_API_URL" "$GIT_COMMIT"' }   // salud + commit
desplegado
    }
    stage('E2E') {                                        // M02
      when { branch 'develop' }
      steps {
        sh '''npx cypress run --project e2e --config baseUrl="$STAGING_WEB_URL" --env
apiUrl="$STAGING_API_URL" \
              --reporter junit --reporter-options "mochaFile=e2e/reports/junit-[hash].xml"'''
      }
      post {
        always {
          junit 'e2e/reports/*.xml'
          archiveArtifacts artifacts: 'e2e/cypress/videos/**, e2e/cypress/screenshots/**',
allowEmptyArchive: true
        }
      }
    }
```

```
    stage('Rendimiento') {                                  // M03, M04: solo etiquetas v*-rc.*
      when { tag pattern: 'v*-rc.*', comparator: 'GLOB' }
      environment {
        DATABASE_URL = "postgresql://calis:calis@postgres-ci:5432/${env.DB_NAME}_perf"
        JWT_SECRET = 'solo-para-la-prueba-de-carga'
        NODE_ENV = 'perf'
      }
      steps {
        sh 'createdb "${DB_NAME}_perf" && npm run -w api db:migrate && npm run -w api
seed:perf'
        sh 'npm run -w api build && (node api/dist/main.js > api.log 2>&1 & echo $! > api.pid)'
        sh './scripts/esperar-salud.sh http://localhost:3000/api/salud'
        sh '''
          jmeter -n -t perf/boletin.jmx  -Jusuarios=100 -Jduracion=600 -l perf/boletin.jtl  -e
-o perf/boletin
          jmeter -n -t perf/planilla.jmx -Jusuarios=100 -Jduracion=600 -l perf/planilla.jtl -e
-o perf/planilla
          jq -e ".Total.pct2ResTime <= 2000" perf/boletin/statistics.json
          jq -e ".Total.errorPct < 1"         perf/boletin/statistics.json
          jq -e ".Total.errorPct < 1"         perf/planilla/statistics.json
        '''
      }
      post {
        always {
          sh 'kill "$(cat api.pid)" || true; dropdb --if-exists "${DB_NAME}_perf"'
          archiveArtifacts artifacts: 'perf/**, api.log', allowEmptyArchive: true
        }
      }
    }
    stage('Aprobar producción') {
      when { tag pattern: 'v\\d+\\.\\d+\\.\\d+', comparator: 'REGEXP' }
      steps {
        input message: '¿Desplegar a producción? Aprueba quien NO creó la etiqueta.',
              ok: 'Aprobar', submitter: 'lujan,galvan'
      }
    }
    stage('Deploy producción') {
      when { tag pattern: 'v\\d+\\.\\d+\\.\\d+', comparator: 'REGEXP' }
      steps {
        withCredentials([string(credentialsId: 'render-hook-prod', variable: 'HOOK')]) {
          sh 'curl -fsS -X POST "$HOOK&ref=$GIT_COMMIT"'
        }
      }
    }
  }
}
```

C.2 Imagen y servicios de Jenkins

```
# jenkins/Dockerfile — imagen del controlador con todo lo que usan las etapas
FROM jenkins/jenkins:lts-jdk17
USER root
RUN apt-get update && apt-get install -y --no-install-recommends \
      curl unzip jq gnupg postgresql-client \
      xvfb libgtk-3-0 libgbm1 libnotify4 libnss3 libxss1 libxtst6 xauth \
 && curl -fsSL https://deb.nodesource.com/setup_22.x | bash - \
 && apt-get install -y nodejs \
```

```
 && curl -fsSL https://archive.apache.org/dist/jmeter/binaries/apache-jmeter-5.6.3.tgz | tar xz
-C /opt \
 && ln -s /opt/apache-jmeter-5.6.3/bin/jmeter /usr/local/bin/jmeter \
 && rm -rf /var/lib/apt/lists/*
#    Dependencias de Cypress: lista de la documentación de Cypress para Debian
#    (los nombres de paquete cambian entre versiones de Debian; verificar al construir).
# SONAR_SCANNER_VERSION: versión vigente en binaries.sonarsource.com
ARG SONAR_SCANNER_VERSION
RUN curl -fsSL -o /tmp/ss.zip \
      "https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-$
{SONAR_SCANNER_VERSION}-linux-x64.zip" \
 && unzip -q /tmp/ss.zip -d /opt && rm /tmp/ss.zip \
 && ln -s /opt/sonar-scanner-${SONAR_SCANNER_VERSION}-linux-x64/bin/sonar-scanner
/usr/local/bin/sonar-scanner
USER jenkins
COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
RUN jenkins-plugin-cli -f /usr/share/jenkins/ref/plugins.txt

# jenkins/plugins.txt
workflow-aggregator
git
github-branch-source
basic-branch-build-strategies
junit
credentials-binding
timestamper
pipeline-stage-view
matrix-auth

# jenkins/docker-compose.yml
services:
  jenkins:
    build: { context: ., args: { SONAR_SCANNER_VERSION: "<versión vigente>" } }
    ports: ["8080:8080"]
    volumes: [jenkins_home:/var/jenkins_home]
    environment:
      STAGING_API_URL: "https://<api-staging>.onrender.com"
      STAGING_WEB_URL: "https://<web-staging>.vercel.app"
    depends_on: [postgres-ci, redis-ci]
  postgres-ci:
    image: postgres:16
    environment: { POSTGRES_USER: calis, POSTGRES_PASSWORD: calis, POSTGRES_DB: calis }
  redis-ci:
    image: redis:7
volumes:
  jenkins_home: {}
```

C.3 Respaldos (única tarea en GitHub Actions)

```
# .github/workflows/backup.yml — M14 (RPO 24 h, RTO 2 h)
name: backup
on:
  schedule:
    - cron: '0 7 * * *'                 # 03:00 en Bolivia (UTC-4)
  workflow_dispatch:
permissions:
  contents: read
jobs:
```

```
  respaldar:
    environment: production
    runs-on: ubuntu-latest
    steps:
      - run: |
          docker run --rm postgres:16 pg_dump "${{ secrets.DATABASE_URL }}" -Fc > db.dump
          gpg --batch --symmetric --cipher-algo AES256 \
              --passphrase "${{ secrets.BACKUP_PASSPHRASE }}" -o "calis-$(date +%F).dump.gpg"
db.dump
          rm db.dump
      - uses: actions/upload-artifact@v4
        with: { name: respaldo-db, path: '*.dump.gpg', retention-days: 30 }
  # El job "restaurar" (workflow_dispatch, cada día 5) descifra el último respaldo,
  # lo restaura con pg_restore en un PostgreSQL 16 vacío y compara el conteo de tablas.
```

C.4 Umbral de cobertura de los módulos críticos

```
// api/jest.config.ts
import type { Config } from 'jest';
const config: Config = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  rootDir: '.',
  testMatch: ['<rootDir>/src/**/*.spec.ts'],
  collectCoverageFrom: ['src/**/*.ts', '!src/**/*.module.ts', '!src/main.ts'],
  coverageReporters: ['lcov', 'text-summary', 'cobertura'],
  coverageThreshold: {                    // M06: el mismo 90 de la Tabla de métricas
    './src/modules/notas/':     { lines: 90 },
    './src/modules/calculo/':   { lines: 90 },
    './src/modules/auditoria/': { lines: 90 },
    './src/modules/auth/':      { lines: 90 },
    './src/modules/familias/':  { lines: 90 },
  },
};
export default config;
```

C.5 SonarQube, commits, hooks y Dependabot

```
# sonar-project.properties
sonar.projectKey=calisfacu
sonar.organization=l001-calisfacu
sonar.sources=api/src,web/src
sonar.tests=api/src,api/test,web/src
sonar.test.inclusions=**/*.spec.ts,**/*.e2e-spec.ts,**/*.test.tsx
sonar.javascript.lcov.reportPaths=api/coverage/lcov.info,web/coverage/lcov.info

// commitlint.config.mjs
export default {
  extends: ['@commitlint/config-conventional'],
  parserPreset: { parserOpts: { issuePrefixes: ['US-', 'CAL-', '#'] } },
  rules: {
    'scope-enum': [2, 'always', ['auth', 'academico', 'notas', 'calculo', 'auditoria',
      'asistencia', 'importacion', 'boletin', 'panel', 'centralizacion', 'notificaciones',
      'web', 'e2e', 'perf', 'ci', 'deps', 'sqap']],
    'references-empty': [2, 'never'],     // todo commit cita US-NN, CAL-NNN o #issue
  },
};
```

```
# .husky/pre-commit      ->  npx lint-staged
# .husky/commit-msg      ->  npx --no -- commitlint --edit "$1"
# .husky/pre-push        ->  npm run -w api test -- --onlyChanged

// .lintstagedrc.json
{ "*.{ts,tsx}": ["eslint --fix --max-warnings=0", "prettier --write"],
  "*.{json,md,yml}": ["prettier --write"] }

# .github/dependabot.yml
version: 2
updates:
  - { package-ecosystem: npm, directory: "/", schedule: { interval: weekly } }
  - { package-ecosystem: github-actions, directory: "/", schedule: { interval: weekly } }
```

Anexo D — Plantillas

D.1 Plantilla de pull request

```
<!-- .github/pull_request_template.md -->
## Historia o defecto
Refs: US-NN / CAL-NNN (tarea T-NNN)

## Qué cambia
-

## Cómo se probó
- [ ] Pruebas nuevas para cada criterio de aceptación tocado
- [ ] Probado en local con datos sintéticos

## Definition of Done
- [ ] Check de Jenkins en verde (Lint, Pruebas, Dependencias, Análisis estático, Build)
- [ ] Sin secretos ni datos reales en el diff
- [ ] Documentación actualizada (SRS, SAD, manual o SQAP) si cambió el comportamiento
- [ ] Revisor asignado según la Sección 6 del SQAP (nadie aprueba su propio PR)
```

D.2 Formulario de defecto

```
# .github/ISSUE_TEMPLATE/defecto.yml
name: Defecto
description: Reportar un defecto (Sección 8 del SQAP)
title: "CAL-NNN: "
labels: ["tipo:defecto", "estado:nuevo"]
body:
  - type: dropdown
    id: severidad
    attributes: { label: Severidad (la fija QA), options: [S1 Crítica, S2 Alta, S3 Media, S4
Baja] }
    validations: { required: true }
  - type: input
    id: modulo
    attributes: { label: Módulo y versión, placeholder: "boletin · v1.0.0-rc.1" }
  - type: input
    id: entorno
    attributes: { label: Entorno, placeholder: "staging · Chrome · Windows 11" }
```

```
  - type: textarea
    id: pasos
    attributes: { label: Pasos para reproducir }
    validations: { required: true }
  - type: textarea
    id: esperado
    attributes: { label: Resultado esperado (cita el criterio o RNF) }
  - type: textarea
    id: observado
    attributes: { label: Resultado observado }
  - type: textarea
    id: evidencia
    attributes: { label: Evidencia (enlace al build de Jenkins, captura o log) }
```

D.3 Formulario de solicitud de cambio de historia

```
# .github/ISSUE_TEMPLATE/cambio-historia.yml
name: Solicitud de cambio de historia
description: Proponer un cambio a una historia o criterio (Sección 10.7 del SQAP)
title: "Cambio: US-NN "
labels: ["tipo:cambio"]
body:
  - type: input
    id: historia
    attributes: { label: Historia afectada, placeholder: "US-02" }
    validations: { required: true }
  - type: dropdown
    id: tipo
    attributes: { label: Tipo de cambio, options: [Nueva, Modificar, Dividir, Descartar,
Cambiar criterio] }
  - type: textarea
    id: cambio
    attributes: { label: Cambio pedido y motivo }
    validations: { required: true }
  - type: input
    id: solicitante
    attributes: { label: Quién lo pide }
  - type: textarea
    id: impacto
    attributes: { label: Impacto (lo completa el Líder), description: "Puntos, sprint, pruebas,
RNF o umbral del SQAP" }
```

D.4 Acta de revisión, inspección o auditoría (`docs/actas/AAAA-MM-DD-<tipo>.md`): fecha y duración; tipo de revisión; artefacto y versión; participantes con su rol; métricas de la inspección (Sección 6.1); hallazgos con ubicación, severidad (mayor o menor) y responsable; decisión (aceptado, aceptado con cambios, rechazado); firmas.

Anexo E — Autoevaluación contra la regla del plan verificable

La última columna la completa un integrante que **no** redactó la sección evaluada (Ejercicio 5 de la Práctica 7): Lujan evalúa las secciones de Galvan y Galvan las de Lujan, según la Tabla de autoría de la Sección 16.

Tabla 36. Autoevaluación del plan

Indicador	Cómo lo cumple este plan	Evaluación del otro integrante
Completo	Están las 16 secciones de IEEE 730-2002, sin "Pendiente"; ninguna se declara "no aplica".	
Medible	Las 15 métricas (M01 a M09 y M11 a M16) tienen fórmula y valor numérico; ningún objetivo dice "rápido", "intuitivo" o "alto".	
Instrumentado	Cada métrica nombra herramienta, condiciones y momento, y la Tabla 10 dice dónde se configura.	
Asignado e independiente	RACI con una A por fila; Galvan no programa módulos críticos; Lujan audita las evidencias de Galvan y verifica sus defectos.	
Consistente	80, 90, 3, 2000, 1, 0, 100, 68, 99,0, 24 h y 2 h aparecen iguales en las Secciones 5, 7 y 12, en el Anexo A, en `matrices-sqap.xlsx` y en el Anexo C.	
Trazable	Cada métrica cita un RNF o un riesgo; cada historia tiene su trazabilidad en el Anexo B; los nueve riesgos altos tienen mitigación con responsable.	
Evidenciable	La Sección 12 da ubicación, formato, retención y responsable de cada evidencia.	

Anexo F — Datos y decisiones pendientes

El plan rige con estos supuestos. Las reglas del cálculo y de la asistencia se resolvieron con las decisiones D1 a D20 de docs/backlog/reglas-ministerio.md (13/10/2026 en adelante); la última columna indica el estado al 30/10/2026. Lo que sigue abierto se registra en la Sección 16 cuando se resuelva.

Tabla 37. Pendientes

Pendiente	Quién lo resuelve	Estado al 30/10/2026	Dónde impacta
Nombre de la unidad educativa piloto y de sus representantes	Líder	Resuelto (30/10/2026): Unidad Educativa Nazareno; directora Fabiana Parrado y profesora Fatima Carreño.	Portada, Secciones 1 y 3

Pendiente	Quién lo resuelve	Estado al 30/10/2026	Dónde impacta
Aprobación de US-17 a US-20 y de dejar los pagos fuera de la v1.0	Docente	Resuelto: el docente aprobó US-17 a US-20 y dejar los pagos fuera (confirmado el 30/10/2026).	Sección 1 y backlog
Regla de la nota anual (¿promedio de los tres trimestres?)	Cliente, con el texto oficial del reglamento	Resuelto: D3 (promedio de los tres trimestres, redondeo D2).	RNF-01, US-02, glosario
¿Se admite 0 en una actividad no presentada? ¿Redondeo 0,5 hacia arriba?	Cliente	Resuelto: D1 (no; mínimo 1) y D2 (0,5 hacia arriba, al final de la suma).	RNF-01, US-01, US-02
¿La licencia cuenta como asistencia? Umbral de "en riesgo"	Cliente	Resuelto: D5 (la licencia cuenta) y D6 (en riesgo: promedio < 51 o asistencia < 80 %, configurable).	US-19, US-06
Absorber la sobrecarga de la semana 1 (horas extra) o pasar US-02 a la semana 2	Equipo	Resuelto: US-02 se mantuvo en la semana 1 (backlog v2.0, Tabla 3).	Sección 1, R09
Proveedor del monitor de disponibilidad y del correo con API HTTP	Galvan	Monitor resuelto (30/10/2026): UptimeRobot. Correo: abierto hasta US-08 y US-12 (Sprint 4).	Secciones 5, 9 y 11